LOVELY PROFESSIONAL UNIVERSITY
School of Computer Science and Engineering

CONTINUOUS ASSESSMENT 2 • PROJECT REPORT

Bank Marketing Analysis and Term Deposit Prediction using Apache Spark (PySpark)

Distributed data processing with RDDs, DataFrames and Spark SQL, and machine learning with Spark MLlib on the UCI Bank Marketing dataset

45,211	17	15	2
RECORDS	ATTRIBUTES	OBJECTIVES	ML MODELS

Submitted in partial fulfilment of the requirements of the course

INT315: CLUSTER COMPUTING

Bachelor of Technology (B.Tech.) in Computer Science and Engineering

SUBMITTED BY

Krishna Singhal
Registration No.: 12307502
Roll No.: RK3M2328B43
B.Tech. CSE | Batch 2023–2027

SUBMITTED TO

Mr. Pradeep Kumar
Course Instructor, INT315
School of Computer Science and Engineering

COURSE DETAILS	
Course Code and Title	INT315 – Cluster Computing
L–T–P / Credits	2–0–2 / 3 credits
Assessment	Continuous Assessment 2 (CA2) – Project Report
Academic Session	2026–27
Course Outcomes Addressed	CO1, CO3, CO4, CO5

PROJECT DETAILS	
Domain	Banking analytics, big data processing, machine learning
Dataset	UCI Bank Marketing Data Set, bank-full.csv (Moro, Cortez and Rita, 2014)
Spark Components Used	RDD API, DataFrame API, Spark SQL, Spark MLlib Pipelines
Models Built	Logistic Regression, Decision Tree Classifier
Tools and Technologies	Apache Spark 4.2.0, PySpark, Python 3.12.3, Ubuntu on WSL
Date of Submission	24 October 2026

Declaration

I, **Krishna Singhal** (Registration Number **12307502**), a student of B.Tech. Computer Science and Engineering at Lovely Professional University, hereby declare that the project report titled **"Bank Marketing Analysis and Term Deposit Prediction using Apache Spark (PySpark)"** is submitted in partial fulfilment of the Continuous Assessment 2 requirement of the course **INT315: Cluster Computing**.

The work presented in this report was carried out by me under the guidance of **Mr. Pradeep Kumar**. The PySpark program, the analysis of its output and the interpretation of results are my own work. The dataset is publicly available and has been cited in the References. Wherever the work of other authors has been used, it has been duly acknowledged.

I further declare that this report has not been submitted, in part or in full, to any other university or institution for the award of any degree, diploma or credit.

Name	Krishna Singhal
Registration Number	12307502
Signature	_____
Date	24 October 2026
Place	Phagwara, Punjab

Certificate

This is to certify that the project report titled **"Bank Marketing Analysis and Term Deposit Prediction using Apache Spark (PySpark)"**, submitted by **Krishna Singhal** (Registration Number **12307502**), is a bona fide record of work carried out under my supervision for the course **INT315: Cluster Computing** during the academic session 2026–27.

The project demonstrates the use of Apache Spark for distributed data processing, including Resilient Distributed Dataset (RDD) operations, DataFrame transformations, Spark SQL queries with joins and aggregations, and machine learning pipelines built with Spark MLlib. To the best of my knowledge, the work is original and fulfils the requirements of the Continuous Assessment 2 component of the course.

Course Instructor	Mr. Pradeep Kumar
Department	School of Computer Science and Engineering
Signature	_____
Date	_____

Acknowledgement

I would like to express my sincere gratitude to **Mr. Pradeep Kumar**, my course instructor for INT315: Cluster Computing, for his guidance throughout this project. His lectures on the Spark architecture, Resilient Distributed Datasets, Spark SQL and Spark MLlib gave me the foundation I needed to design and carry out this analysis. His emphasis on understanding why a distributed operation behaves as it does, rather than only how to call it, shaped the way I approached each objective in this report.

I am thankful to the **School of Computer Science and Engineering, Lovely Professional University**, for providing a curriculum that combines the theory of big data systems with hands-on practical work. The laboratory sessions on Spark SQL, MLlib and the PySpark shell were directly useful in building this project.

I acknowledge the authors of the Bank Marketing dataset, **S. Moro, P. Cortez and P. Rita**, and the **UCI Machine Learning Repository** for making the data publicly available for research and education. I also thank the open-source community behind **Apache Spark**, whose documentation and examples were an important reference.

I am grateful to my classmates for the discussions on feature engineering, class imbalance and model evaluation, which helped me refine my approach. Finally, I thank my family for their constant encouragement and support during the course of this work.

Krishna Singhal

Registration Number: 12307502

B.Tech. Computer Science and Engineering, Lovely Professional University

Abstract

This project uses Apache Spark, through its Python interface PySpark, to analyse 45,211 records of a Portuguese bank's direct telephone marketing campaigns and to predict whether a client will subscribe to a term deposit. The work is organised as fifteen objectives that progress from data loading and quality checks, through exploratory analysis with RDDs, DataFrames and Spark SQL, to a complete machine learning pipeline built with Spark MLlib.

The dataset contained no null cells and no exact duplicate rows, but several categorical columns used the literal value "unknown", most notably *poutcome* (36,959 records) and *contact* (13,020 records). The target variable is strongly imbalanced: only 5,289 clients (11.70%) subscribed, while 39,922 (88.30%) did not.

The exploratory analysis revealed clear patterns. Clients aged 60 and above subscribed at 33.63%, nearly three times the overall rate. Students (28.68%) and retired clients (22.79%) were the most responsive occupations, while blue-collar workers (7.27%) were the least. Clients without any housing or personal loan subscribed at 18.22%. Cellular contact (14.92%) outperformed unknown contact methods (4.07%). The strongest single signal was the outcome of the previous campaign: clients whose earlier campaign had succeeded subscribed at 64.73%.

For prediction, fifteen input attributes (nine categorical and six numerical) were selected. The call *duration* was deliberately excluded because it is only known after a call ends and would leak the outcome. Categorical inputs were encoded with StringIndexer and OneHotEncoder and combined with VectorAssembler into a 59-dimensional feature vector, with the preprocessing fitted only on the 36,194-record training split. Logistic Regression and a Decision Tree were trained and evaluated on a 9,017-record test split.

Logistic Regression achieved 89.36% accuracy and the higher ROC-AUC (0.7788), but recalled only 7.96% of subscribers (80 of 1,005). The Decision Tree achieved 89.61% accuracy, more than double the subscriber recall (16.92%, 170 of 1,005) and the higher F1-score (0.2662), while both models had almost the same PR-AUC (0.3973 and 0.3948). Because a trivial model that always predicts "no" already scores 88.85% accuracy on the test set, the report argues that accuracy alone is misleading for this problem and that precision, recall, F1-score and PR-AUC must guide model choice. The report closes with business recommendations, limitations and directions for future work, such as class weighting, threshold tuning and ensemble methods.

Keywords: Apache Spark, PySpark, RDD, Spark SQL, Spark MLlib, bank marketing, term deposit, class imbalance, logistic regression, decision tree, big data analytics

Table of Contents

No.	Title
–	Declaration
–	Certificate
–	Acknowledgement
–	Abstract
–	List of Figures
–	List of Tables
–	List of Abbreviations
1	**Introduction**
1.1	Background and Motivation
1.2	Problem Statement
1.3	Objectives of the Project
1.4	Scope of the Project
1.5	Organisation of the Report
2	**Background: Big Data Processing with Apache Spark**
2.1	Limitations of Hadoop MapReduce
2.2	Apache Spark and Its Architecture
2.3	Spark vs Hadoop Ecosystem
2.4	Resilient Distributed Datasets (RDDs)
2.5	DataFrames and Spark SQL
2.6	Spark MLlib and ML Pipelines
2.7	Classification Algorithms Used
2.8	Evaluation Metrics for Imbalanced Classification
3	**Dataset Description**
3.1	Source and Context
3.2	Attribute Description
3.3	Special Values and Data Characteristics
3.4	Dataset Files, Provenance and Licence
3.5	Distribution of Categorical Attributes
3.6	Numerical Attributes by Outcome
3.7	Data Consistency Checks

No.	Title
3.8	Suitability of the Dataset for Spark
4	**Methodology and System Setup**
4.1	Software Environment
4.2	Overall Workflow
4.3	Program Structure and Helper Functions
4.4	Spark Configuration Choices
5	**Data Loading, Quality and Exploratory Analysis**
5.1	Objective 1: Load the Dataset and Inspect Its Structure
5.2	Objective 2: Null Values, Duplicates and “unknown” Values
5.3	Objective 3: Descriptive Statistics
5.4	Objective 4: Target Distribution and Class Imbalance
5.5	Objective 5: Occupation Counts using RDD Operations
6	**Segment Analysis with DataFrames and Spark SQL**
6.1	Objective 6: Subscription Rate by Age Group
6.2	Objective 7: Occupation and Education using Spark SQL
6.3	Objective 8: Housing and Personal Loans
6.4	Objective 9: Contact Method and Campaign Month
6.5	Objective 10: Previous Campaign Outcome using an SQL LEFT JOIN
7	**Feature Engineering and the ML Pipeline**
7.1	Objective 11: Feature Selection and Train–Test Split
7.2	Objective 12: Encoding and Feature Assembly with a Spark ML Pipeline
8	**Model Training and Evaluation**
8.1	The Common Evaluation Helper
8.2	Objective 13: Logistic Regression
8.3	Objective 14: Decision Tree
8.4	Objective 15: Model Comparison and Baseline
9	**Discussion and Business Insights**
9.1	Consolidated Findings
9.2	Recommendations for the Marketing Team
9.3	Mapping to Course Outcomes
10	**Limitations and Future Scope**

No.	Title
10.1	Limitations
10.2	Future Scope
11	**Conclusion**
–	**References**
A	**Appendix A: Complete Console Output**
B	**Appendix B: Complete Source Code**

List of Figures

Figure	Title
4.1	End-to-end workflow of the project
5.1	Distribution of the target variable (subscribed vs not subscribed)
5.2	Customer count by occupation (RDD map and reduceByKey)
6.1	Subscription rate by age group
6.2	Subscription rate by occupation (Spark SQL)
6.3	Subscription rate and call volume by campaign month
6.4	Subscription rate by outcome of the previous campaign
8.1	Confusion matrices of Logistic Regression and Decision Tree
8.2	Comparison of evaluation metrics for both classifiers

List of Tables

Table	Title
1.1	Project objectives and the Spark component used for each
2.1	Comparison of Hadoop MapReduce and Apache Spark
2.2	RDD transformations and actions used in this project
2.3	Structure of a binary confusion matrix
3.1	Description of the 17 attributes in the dataset
3.2	Files in the Bank Marketing dataset archive
3.3	Category counts and subscription rates
3.4	Mean of numerical attributes by target class
4.1	Software environment
5.1	Inferred schema of the dataset
5.2	Count of “unknown” values per categorical column
5.3	Descriptive statistics of numerical attributes
5.4	Target class distribution
5.5	Customers per occupation (RDD output)
6.1	Subscription rate by age group
6.2	Subscription rate by education level
6.3	Subscription rate by housing and personal loan status
6.4	Subscription rate by contact method
6.5	Subscription rate by previous campaign outcome (LEFT JOIN)
7.1	Input features selected for modelling
7.2	Train/test split and label distribution
7.3	Layout of the 59-dimensional feature vector
8.1	Hyperparameters of the two classifiers
8.2	Model comparison on the test set
8.3	Confusion matrix counts
9.1	Consolidated findings from all fifteen objectives
9.2	Mapping of project work to INT315 course outcomes

List of Abbreviations

Abbreviation	Full Form
API	Application Programming Interface
AUC	Area Under the Curve
CSV	Comma-Separated Values
DAG	Directed Acyclic Graph
DT	Decision Tree
EDA	Exploratory Data Analysis
FN / FP	False Negative / False Positive
HDFS	Hadoop Distributed File System
JVM	Java Virtual Machine
LR	Logistic Regression
ML / MLlib	Machine Learning / Spark Machine Learning Library
PR	Precision–Recall
RDD	Resilient Distributed Dataset
ROC	Receiver Operating Characteristic
SQL	Structured Query Language
TN / TP	True Negative / True Positive
UCI	University of California, Irvine
WSL	Windows Subsystem for Linux

Chapter 1: Introduction

1.1 Background and Motivation

Banks depend on deposits to fund their lending activities. A **term deposit** (also called a fixed deposit) is a product in which a client places money with the bank for a fixed period at an agreed interest rate. Because the money cannot be withdrawn freely before maturity, term deposits give the bank a predictable and relatively cheap source of funds. Attracting new term deposits is therefore a recurring commercial goal.

One of the most common ways to promote term deposits is **direct marketing**, in which bank staff call existing or prospective clients on the telephone and offer the product. Direct marketing is effective but expensive. Every call costs staff time, and calling clients who have no interest can harm the client relationship. A bank that can predict which clients are likely to subscribe can concentrate its effort on them, reduce the number of wasted calls and improve the return on each campaign.

Modern banks collect large volumes of data about their clients and about every contact made during a campaign. As the number of clients, campaigns and attributes grows, analysing this data on a single machine with traditional tools becomes slow. Distributed processing frameworks such as **Apache Spark** were designed for exactly this situation. Spark splits data into partitions, processes the partitions in parallel across the cores of a machine or the nodes of a cluster, and keeps intermediate results in memory to avoid repeated disk access.

This project applies Apache Spark to the well-known Bank Marketing dataset from the UCI Machine Learning Repository. Although the dataset (45,211 rows) fits comfortably in memory on a laptop, the same PySpark code would run without modification on a multi-node cluster processing millions of records. The project therefore serves two purposes: it answers a realistic business question, and it demonstrates the core Spark components covered in INT315, namely RDDs, DataFrames, Spark SQL and Spark MLlib.

1.2 Problem Statement

Given the demographic, financial and campaign-related attributes of bank clients contacted during direct marketing campaigns, the project aims to:

1. Understand the structure and quality of the data and the factors associated with a client subscribing to a term deposit.
2. Build and evaluate machine learning classifiers, using Apache Spark, that predict whether a client will subscribe (*y = yes*) or not (*y = no*), using only information that would be available **before** the call is made.

The prediction task is a **binary classification problem with strong class imbalance**, because only about one client in nine subscribes. A useful model must therefore do more than predict the majority class; it must identify a meaningful share of the actual subscribers.

1.3 Objectives of the Project

The project is divided into fifteen objectives. Each objective is implemented as a separate, clearly labelled block in the PySpark program, printing the problem statement, the output and a one-line conclusion.

Table 1.1: Project objectives and the Spark component used for each

No.	Objective	Spark component used
1	Load the dataset and identify the number of records, columns and data types	DataFrame reader, schema inference, caching
2	Identify null values, exact duplicate rows and "unknown" categorical values	DataFrame aggregations
3	Calculate descriptive statistics for numerical attributes	`summary()`, `agg()`
4	Calculate subscriber and non-subscriber counts and identify class imbalance	`groupBy`, label creation
5	Count customers in each occupation using RDD `map` and `reduceByKey`	RDD API
6	Compare subscription rates across age groups	`when/otherwise`, `groupBy`
7	Compare subscription rates by occupation and education using Spark SQL	Temporary views, SQL queries
8	Compare subscription rates for combinations of housing and personal loans	Multi-column `groupBy`
9	Compare subscription rates across contact methods and campaign months	DataFrame aggregations
10	Attach previous-outcome descriptions with an SQL LEFT JOIN and compare rates	Spark SQL joins
11	Select features and split the data into about 80% training and 20% testing	`randomSplit`, leakage control
12	Encode categorical variables and assemble a feature vector	MLlib Pipeline, StringIndexer, OneHotEncoder, VectorAssembler
13	Train and evaluate a Logistic Regression classifier	MLlib LogisticRegression
14	Train and evaluate a Decision Tree classifier	MLlib DecisionTreeClassifier
15	Compare both models using accuracy, ROC-AUC, PR-AUC, precision, recall, F1 and confusion matrices	MLlib evaluators, pivot tables

1.4 Scope of the Project

The scope of this project covers:

- Batch processing of the full Bank Marketing dataset (`bank-full.csv`, 45,211 records, 17 attributes) using PySpark in local mode.
- Data quality assessment, descriptive statistics and segment-level analysis of subscription rates.
- Use of three Spark programming interfaces: the low-level RDD API, the DataFrame API and Spark SQL.
- Construction of a reusable Spark MLlib preprocessing pipeline and training of two classifiers covered in Unit VI of the syllabus.
- Evaluation with metrics suitable for imbalanced data, and a comparison against a majority-class baseline.

The following topics are outside the scope of this project: real-time streaming with Kafka, graph analytics with GraphX, hyperparameter tuning with cross-validation, and deployment of the model into a production system. These are discussed as future work in Chapter 10.

1.5 Organisation of the Report

Chapter 2 reviews the theory behind Spark, RDDs, Spark SQL, MLlib and the evaluation metrics used. Chapter 3 describes the dataset and its attributes. Chapter 4 explains the software environment, the workflow and the structure of the program. Chapters 5 to 8 present the fifteen objectives in order; for each objective the report gives the problem statement, the approach, the relevant code, the actual output and an interpretation. Chapter 9 consolidates the findings into business insights and maps the work to the course outcomes. Chapter 10 lists the limitations and future scope, and Chapter 11 concludes the report. Appendix A contains the complete console output and Appendix B contains the complete source code, reproduced exactly as submitted.

Chapter 2: Background – Big Data Processing with Apache Spark

2.1 Limitations of Hadoop MapReduce

Hadoop MapReduce was the first widely adopted framework for processing very large datasets across clusters of commodity machines. A MapReduce job has two phases. In the **map** phase, each input record is transformed into key–value pairs. In the **reduce** phase, all values sharing a key are grouped and combined. Hadoop handles partitioning, scheduling and fault tolerance automatically, which made distributed computing accessible to many organisations.

However, MapReduce has several limitations that matter for analytics and machine learning:

- **Disk-based intermediate storage.** The output of every map and reduce stage is written to HDFS or local disk. A pipeline of several jobs therefore reads and writes the full data many times, which is slow.
- **Poor support for iterative algorithms.** Machine learning algorithms such as logistic regression repeat the same computation over the same data many times. In MapReduce each iteration is a new job that re-reads the data from disk.
- **Rigid programming model.** Every computation must be expressed as a sequence of map and reduce steps. Joins, filters and multi-step aggregations require verbose code or additional tools such as Hive or Pig.
- **Batch only.** MapReduce is designed for batch processing with high latency. It is not suitable for interactive queries or real-time stream processing.

2.2 Apache Spark and Its Architecture

Apache Spark is an open-source, general-purpose distributed computing engine originally developed at the University of California, Berkeley. Its central idea is to keep working data **in memory** across operations, so that iterative and interactive workloads avoid repeated disk access. Spark provides high-level APIs in Scala, Java, Python and R, and a unified stack of libraries: Spark SQL for structured data, MLlib for machine learning, GraphX for graph processing and Structured Streaming for real-time data.

A Spark application has the following main components:

- **Driver program.** The process that runs the user's `main` code and creates the `SparkSession`. The driver converts the user's operations into a **Directed Acyclic Graph (DAG)** of stages and tasks.
- **Cluster manager.** The service that allocates resources. Spark can run on its own standalone manager, YARN, Kubernetes, or in **local mode**, where the driver and executors share a single machine. This project runs in local mode.
- **Executors.** Worker processes that run tasks on partitions of the data and store cached partitions in memory.
- **Tasks and partitions.** Data is split into partitions; each task processes one partition. The number of partitions controls the degree of parallelism.

Spark uses **lazy evaluation**. Transformations such as `filter`, `select`, `groupBy` or `map` only add steps to the plan. Computation is triggered when an **action** such as `count`, `show`, `collect` or `first` is called. Lazy evaluation lets Spark optimise the whole plan before running it, for example by combining filters or pushing them closer to the data source.

2.3 Spark vs Hadoop Ecosystem

Table 2.1: Comparison of Hadoop MapReduce and Apache Spark

Aspect	Hadoop MapReduce	Apache Spark
Processing model	Batch, two-stage map and reduce	General DAG of transformations
Intermediate data	Written to disk after every stage	Kept in memory where possible
Speed for iterative jobs	Slow, each iteration re-reads disk	Much faster due to in-memory caching
Ease of programming	Verbose Java code	Concise APIs in Python, Scala, Java, R
SQL support	Through Hive	Built-in Spark SQL
Machine learning	Through Mahout (largely retired)	Built-in MLlib
Streaming	Not supported natively	Structured Streaming, Kafka integration
Graph processing	Not supported natively	GraphX
Fault tolerance	Re-execution from replicated HDFS data	Recomputation using RDD lineage
Storage	HDFS	Any source: HDFS, local files, S3, databases

Spark does not replace the whole Hadoop ecosystem. It is often deployed on Hadoop clusters, reading data from HDFS and using YARN as its cluster manager, while replacing MapReduce as the processing engine.

2.4 Resilient Distributed Datasets (RDDs)

The **Resilient Distributed Dataset** is Spark's fundamental data abstraction. An RDD is an immutable, partitioned collection of records that can be processed in parallel. Its main features are:

- **Resilient.** If a partition is lost because an executor fails, Spark rebuilds it from its **lineage**, the recorded sequence of transformations that produced it.
- **Distributed.** The records are split into partitions spread across the executors.
- **Immutable.** A transformation never changes an existing RDD; it produces a new one.
- **Lazily evaluated** and **cacheable** in memory with `cache()` or `persist()`.

RDD operations are of two kinds. **Transformations** return a new RDD and are lazy; **actions** return a result to the driver or write output and trigger execution. Transformations are further classified as **narrow** (each output partition depends on one input partition, as in `map`) or **wide** (data must be shuffled across partitions, as in `reduceByKey`).

Table 2.2: RDD transformations and actions used in this project

Operation	Type	Role in this project
`df.select("job").rdd`	Conversion	Converts a DataFrame column to an RDD of Row objects
`map(lambda row: (row["job"], 1))`	Narrow transformation	Emits a (job, 1) pair for every customer
`reduceByKey(lambda`	Wide transformation	Sums the counts for each job across

Operation	Type	Role in this project
`a, b: a + b)`		partitions
`toDF([...])`	Conversion	Converts the result back to a DataFrame
`first()`, `show()`	Actions	Trigger execution and display results

The `map` and `reduceByKey` pattern in Objective 5 is the Spark equivalent of the classic MapReduce word-count program. `reduceByKey` combines values locally inside each partition before shuffling, which reduces the amount of data sent across the network.

2.5 DataFrames and Spark SQL

A **DataFrame** is a distributed collection of rows organised into named columns with a schema, similar to a table in a relational database. DataFrames are built on top of RDDs but carry schema information, which allows Spark's **Catalyst optimiser** to analyse and rewrite queries, and the **Tungsten** execution engine to store data in a compact binary format. For most structured workloads, DataFrames are both easier to write and faster than raw RDDs.

Spark SQL lets users query DataFrames with standard SQL. A DataFrame is registered as a temporary view with `createOrReplaceTempView`, and queries are run with `spark.sql(...)`. Spark SQL supports `SELECT`, `WHERE`, `GROUP BY`, `ORDER BY`, aggregate functions such as `COUNT`, `AVG` and `SUM`, `LIKE` with wildcards, and all common join types (`INNER`, `LEFT`, `RIGHT`, `FULL`). SQL queries and DataFrame code compile to the same optimised plan, so the choice between them is a matter of readability. RDDs can be converted to DataFrames with `toDF()` or `spark.createDataFrame()`, both of which are used in this project.

2.6 Spark MLlib and ML Pipelines

MLlib is Spark's scalable machine learning library. The modern DataFrame-based API (`pyspark.ml`) is built around a few key concepts:

- **Transformer.** An object that converts one DataFrame into another, for example by adding a column of predictions. A trained model is a transformer.
- **Estimator.** An algorithm that is **fitted** on a DataFrame to produce a transformer. `StringIndexer` and `LogisticRegression` are estimators.
- **Pipeline.** An ordered sequence of stages (estimators and transformers). Fitting a pipeline fits each stage in turn and returns a **PipelineModel** that can be applied to new data.
- **Feature vector.** MLlib algorithms expect all inputs in a single column of type `Vector`. Sparse vectors store only non-zero values, which is efficient for one-hot encoded data.

The feature transformers used in this project are:

- **StringIndexer**, which maps each category of a string column to a numeric index, with the most frequent category receiving index 0.
- **OneHotEncoder**, which converts each index into a binary vector so that the model does not treat categories as ordered numbers.
- **VectorAssembler**, which concatenates numerical columns and encoded vectors into the single `features` column.

2.7 Classification Algorithms Used

Logistic Regression models the probability that the label equals 1 as a logistic (sigmoid) function of a weighted sum of the features:

$$P(y = 1 \mid x) = \frac{1}{1 + e^{-(\beta_0 + \beta_1 x_1 + \dots + \beta_n x_n)}}$$

The weights are learned by minimising the log-loss on the training data. A regularisation term, controlled by `regParam`, penalises large weights to reduce overfitting. By default a client is predicted as a subscriber when the predicted probability exceeds 0.5. Logistic Regression is fast, interpretable and produces well-ranked probabilities, but it can only represent linear decision boundaries in the feature space.

A **Decision Tree** splits the data recursively. At each node it chooses the feature and threshold that best separate the classes, measured by an impurity criterion such as Gini impurity:

$$Gini = 1 - \sum_{k} p_k^{2}$$

where p_k is the proportion of class k in the node. Splitting stops when the tree reaches `maxDepth` or no split improves purity. Each leaf predicts the majority class of its training records, and its class proportions are reported as the probability. Decision trees capture non-linear relationships and interactions between features, and are easy to explain as rules, but deep trees overfit easily.

2.8 Evaluation Metrics for Imbalanced Classification

For a binary classifier, every test record falls into one of four cells of the **confusion matrix**:

Table 2.3: Structure of a binary confusion matrix

	Predicted No (0)	Predicted Yes (1)
Actual No (0)	True Negative (TN)	False Positive (FP)
Actual Yes (1)	False Negative (FN)	True Positive (TP)

From these counts the following metrics are derived:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

$$\text{Precision} = \frac{TP}{TP + FP} \qquad \text{Recall} = \frac{TP}{TP + FN}$$

$$F_1 = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

Accuracy is misleading when one class dominates: a model that predicts “no” for everyone would be correct for roughly 88% of clients in this dataset while finding no subscribers at all. **Precision** answers “of the clients the model flagged, how many actually subscribed?” and **recall** answers “of all actual subscribers, how many did the model find?”. **F1** is their harmonic mean.

ROC-AUC is the area under the curve of true positive rate against false positive rate across all thresholds. It measures how well the model ranks subscribers above non-subscribers, with 0.5 meaning random ranking and 1.0 meaning perfect ranking. **PR-AUC** is the area under the precision–recall curve. For imbalanced data, PR-AUC is more informative than ROC-AUC because it focuses on the minority class; a random classifier’s PR-AUC equals the positive-class proportion (about 0.11 here).

Chapter 3: Dataset Description

3.1 Source and Context

The dataset used in this project is the **Bank Marketing Data Set** published in the UCI Machine Learning Repository by Sérgio Moro, Paulo Cortez and Paulo Rita. It records the direct marketing campaigns of a Portuguese retail bank, conducted mainly through telephone calls between May 2008 and November 2010. Often more than one contact with the same client was required to find out whether the client would subscribe to the product.

The downloaded archive contains two versions of the data:

- **bank.zip**, containing `bank-full.csv` (45,211 rows, all examples, ordered by date), `bank.csv` (4,521 rows, a 10% random sample) and `bank-names.txt` (the attribute description).
- **bank-additional.zip**, a later version with 41,188 rows and additional socio-economic context attributes.

This project uses **bank-full.csv**, the complete original dataset with 17 attributes. The file uses a semicolon (`;`) as the field separator and has a header row, which is why the program reads it with `.option("sep", ";")` and `.option("header", True)`.

3.2 Attribute Description

The 17 attributes fall into four groups: client demographics, client financial status, details of the last contact in the current campaign, and history from earlier campaigns. The final attribute, *y*, is the target.

Table 3.1: Description of the 17 attributes in the dataset

No.	Attribute	Type	Group	Description
1	age	Integer	Client	Age of the client in years
2	job	Categorical	Client	Type of job: admin., blue-collar, entrepreneur, housemaid, management, retired, self-employed, services, student, technician, unemployed, unknown
3	marital	Categorical	Client	Marital status: married, single, divorced (divorced includes widowed)
4	education	Categorical	Client	Education level: primary, secondary, tertiary, unknown
5	default	Binary	Financial	Does the client have credit in default? (yes/no)
6	balance	Integer	Financial	Average yearly balance in euros
7	housing	Binary	Financial	Does the client have a housing loan? (yes/no)
8	loan	Binary	Financial	Does the client have a personal loan? (yes/no)
9	contact	Categorical	Last contact	Contact communication type: cellular, telephone, unknown
10	day	Integer	Last contact	Day of the month of the last contact
11	month	Categorical	Last contact	Month of the last contact (jan to dec)

No.	Attribute	Type	Group	Description
12	duration	Integer	Last contact	Duration of the last contact in seconds
13	campaign	Integer	Current campaign	Number of contacts made during this campaign for this client, including the last
14	pdays	Integer	Previous campaigns	Days since the client was last contacted in a previous campaign; −1 means not previously contacted
15	previous	Integer	Previous campaigns	Number of contacts before this campaign for this client
16	poutcome	Categorical	Previous campaigns	Outcome of the previous campaign: success, failure, other, unknown
17	y	Binary	Target	Has the client subscribed to a term deposit? (yes/no)

3.3 Special Values and Data Characteristics

Several characteristics of the data influenced the design of the analysis:

- **"unknown" instead of nulls.** Missing categorical information is recorded as the string "unknown" rather than as a null value. A simple null check therefore reports zero missing values, and a separate check for "unknown" is needed (Objective 2).
- **pdays = −1.** A value of −1 in *pdays* is a code meaning the client was not contacted in a previous campaign. It is not a real number of days, so it must be interpreted carefully. Because more than 75% of clients have this value, the 25th, 50th and 75th percentiles of *pdays* are all −1.
- **duration leaks the target.** The call duration is strongly associated with the outcome; a call lasting zero seconds always ends with *y = no*. However, the duration is only known after the call has ended, when the outcome is also known. The dataset authors themselves recommend excluding it from realistic predictive models. This project includes it in descriptive statistics but excludes it from the model (Objective 11).
- **Class imbalance.** Only about 11.7% of records have *y = yes*. This affects both the interpretation of accuracy and the behaviour of the classifiers.
- **Ordered by date.** The rows of `bank-full.csv` are ordered from May 2008 to November 2010. A random train/test split therefore mixes time periods; a strictly time-based evaluation would hold out the latest months instead. This is noted as a limitation in Chapter 10.
- **Skewed numerical distributions.** *balance*, *campaign*, *duration* and *previous* are heavily right-skewed with extreme maximum values (for example, a balance of 102,127 euros and 275 previous contacts), which matters for linear models.

3.4 Dataset Files, Provenance and Licence

The dataset was created by **Paulo Cortez** (University of Minho, Portugal) and **Sérgio Moro** (ISCTE – University Institute of Lisbon) in 2012, and is hosted by the UCI Machine Learning Repository as dataset number 222. It was first described in Moro, Laureano and Cortez (2011), who analysed it with the CRISP-DM data mining methodology, and later in Moro, Cortez and Rita (2014). The data is public and may be used for research and education, provided the original paper is cited.

Table 3.2: Files in the Bank Marketing dataset archive

File	Rows	Attributes	Content and intended use
bank-full.csv	45,211	16 inputs + y	All examples, ordered by date from May 2008 to November 2010. **Used in this project.**
bank.csv	4,521	16 inputs + y	A random 10% sample of bank-full.csv, provided for algorithms that are slow on the full data (for example SVM)
bank-names.txt	–	–	Attribute descriptions, value lists and citation request
bank-additional-full.csv	41,188	20 inputs + y	Later version (May 2008 to November 2010) adding five socio-economic indicators such as the Euribor 3-month rate and the employment variation rate
bank-additional.csv	4,119	20 inputs + y	A random 10% sample of bank-additional-full.csv

The full file is 4.6 MB on disk. Each row is **one contact with one client** in the current campaign, described by the client's profile at that moment, the details of the last call and the history of earlier campaigns. The target *y* records whether that client finally subscribed to a term deposit.

3.5 Distribution of Categorical Attributes

The table below gives, for every categorical attribute not already covered by Objectives 5 and 9, the number of clients in each category, its share of the dataset, and the share of those clients who subscribed. Job and month are covered in Chapters 5 and 6.

Table 3.3: Category counts and subscription rates (overall rate 11.70%)

Attribute	Category	Clients	Share	Subscription rate
marital	married	27,214	60.19%	10.12%
marital	single	12,790	28.29%	14.95%
marital	divorced (incl. widowed)	5,207	11.52%	11.95%
education	secondary	23,202	51.32%	10.56%
education	tertiary	13,301	29.42%	15.01%
education	primary	6,851	15.15%	8.63%
education	unknown	1,857	4.11%	13.57%
default	no	44,396	98.20%	11.80%
default	yes	815	1.80%	6.38%
housing	yes	25,130	55.58%	7.70%
housing	no	20,081	44.42%	16.70%
loan	no	37,967	83.98%	12.66%
loan	yes	7,244	16.02%	6.68%
contact	cellular	29,285	64.77%	14.92%

Attribute	Category	Clients	Share	Subscription rate
contact	unknown	13,020	28.80%	4.07%
contact	telephone	2,906	6.43%	13.42%
poutcome	unknown	36,959	81.75%	9.16%
poutcome	failure	4,901	10.84%	12.61%
poutcome	other	1,840	4.07%	16.68%
poutcome	success	1,511	3.34%	64.73%

Three features of this table matter for modelling. First, **default** is almost constant: 98.20% of clients have no credit in default, so the attribute carries little information for most clients. Second, the typical client is **married with secondary education and a housing loan**, so the majority categories have below-average rates and the minority categories hold most of the signal. Third, the rare **poutcome = success** category covers only 3.34% of clients but has the highest rate of any category in the dataset.

3.6 Numerical Attributes by Outcome

Comparing the average of each numerical attribute for subscribers and non-subscribers shows which attributes separate the two groups.

Table 3.4: Mean of numerical attributes by target class

Attribute	Non-subscribers (y = no)	Subscribers (y = yes)	Difference
age (years)	40.84	41.67	+0.83
balance (€)	1,303.71	1,804.27	+500.56
campaign (contacts)	2.85	2.14	−0.71
previous (contacts)	0.50	1.17	+0.67
duration (seconds)	221.18	537.29	+316.11

Subscribers have higher balances, were contacted **fewer** times in the current campaign, and had **more** contacts in earlier campaigns. The average age is almost the same in both groups, even though Objective 6 showed large differences between age bands: the effect of age is U-shaped, so very young and very old clients cancel out in the mean. The very large duration gap (537 vs 221 seconds) confirms why duration must be excluded from a model that predicts before the call: long calls are a consequence of interest, not a cause of it.

The numerical attributes are also strongly **right-skewed**. The skewness coefficient is 0.68 for age, 3.14 for duration, 4.90 for campaign, 8.36 for balance and 41.85 for previous, where 0 means symmetric. 1,196 clients were contacted more than 10 times in the same campaign, 3,766 clients have a negative balance and 3,514 have a balance of exactly zero.

3.7 Data Consistency Checks

Several cross-checks between related attributes were carried out to confirm the data is internally consistent:

- **pdays and previous agree.** Exactly 36,954 clients have *pdays* = −1, and exactly the same 36,954 have *previous* = 0, so both attributes record “never contacted before” in the same way.

- **poutcome is almost aligned with pdays.** 36,959 clients have *poutcome* = unknown, five more than the 36,954 never contacted. These five clients were contacted before but their previous outcome was not recorded.
- **Zero-length calls.** Three calls have *duration* = 0, and all three ended with *y* = no, matching the dataset authors' note that a zero duration always means no subscription.
- **Value ranges are plausible.** Ages lie between 18 and 95, *day* between 1 and 31, and every month and category value matches the lists given in bank-names.txt.

3.8 Suitability of the Dataset for Spark

At 45,211 rows and 4.6 MB, the dataset is small by big data standards and would fit in memory on any laptop. It was chosen for this course project because it is realistic, well documented and widely used as a benchmark, which makes the results easy to verify. Its structure is typical of real enterprise data: a mix of numerical and categorical attributes, hidden missing values, strong class imbalance and a leakage-prone attribute. The PySpark program handles all of these with distributed operations that do not depend on the data size, so the same code could process the records of a national bank with millions of clients by running on a cluster instead of in local mode.

Chapter 4: Methodology and System Setup

4.1 Software Environment

The project was developed on a Windows laptop using the **Windows Subsystem for Linux (WSL)**, which provides an Ubuntu environment where Spark runs natively. The dataset was stored on the Windows file system and accessed from WSL through the `/mnt/c/...` mount, which is why the input path in the program begins with `file:///mnt/c/Users/krish/...`. The `file://` prefix tells Spark to read from the local file system rather than from HDFS.

Table 4.1: Software environment

Component	Version / Detail	Purpose
Operating system	Ubuntu on WSL (Windows host)	Linux runtime for Spark
Java	OpenJDK (JVM)	Spark runs on the Java Virtual Machine
Apache Spark	4.2.0, local mode (local[*])	Distributed processing engine
PySpark	4.2.0	Python API for Spark
Python	3.12.3	Programming language
Spark modules	`pyspark.sql`, `pyspark.ml`	DataFrames, SQL, MLlib
Execution	PySpark interactive shell	Program pasted and run with `exec(r'''...''')`
Dataset	`bank-full.csv`, 45,211 rows, 4.6 MB	Input data

The program was run inside the **PySpark interactive shell**. In the shell a `SparkSession` named `spark` already exists, and `SparkSession.builder.getOrCreate()` returns that existing session instead of creating a new one. The whole program is wrapped in `exec(r'''...''')` so that it can be pasted into the shell as a single statement. This avoids the indentation errors that occur when multi-line blocks are pasted line by line, and the raw string prefix `r` ensures that backslashes are not treated as escape characters. For the same reason the program ends by unpersisting cached DataFrames but does **not** call `spark.stop()`, so the shell remains usable afterwards.

4.2 Overall Workflow

The fifteen objectives form a single pipeline that moves from raw data to an evaluated model. Figure 4.1 shows how the stages connect.

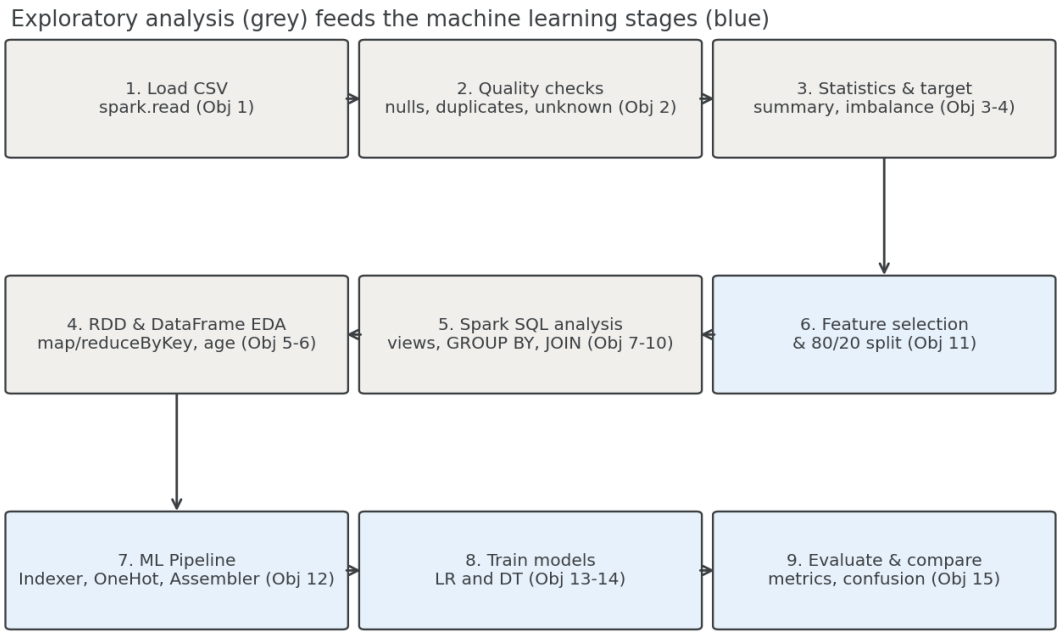


Figure 4.1: End-to-end workflow of the project. Grey boxes are exploratory stages; blue boxes are machine learning stages.

The workflow follows a standard data science process:

1. **Data ingestion.** Read the semicolon-separated CSV into a cached DataFrame with schema inference.
2. **Data understanding.** Check data quality, compute descriptive statistics and examine the target distribution.
3. **Exploratory analysis.** Use RDDs, DataFrames and Spark SQL to compare subscription rates across client segments.
4. **Data preparation.** Select leak-free features, split the data into training and test sets, and build an encoding pipeline fitted on the training data only.
5. **Modelling.** Train Logistic Regression and Decision Tree classifiers on identical training data.
6. **Evaluation.** Score both models on the same test set and compare them against a majority-class baseline.

4.3 Program Structure and Helper Functions

The program is a single script divided into fifteen clearly labelled blocks, one per objective. Two small helper functions give every objective a consistent printed layout:

```
def heading(number, problem):
    print("\n" + "=" * 75)
    print("OBJECTIVE", number)
    print("=" * 75)
    print("PROBLEM STATEMENT:", problem)
    print("\nSOLUTION / OUTPUT:")


def conclusion(text):
    print("\nSOLUTION IN WORDS:", text)
    print("-" * 75)
```

`heading()` prints the objective number and its problem statement before any output, and `conclusion()` prints a plain-language summary afterwards. The conclusions are built with **f-strings**

from values computed at run time (for example `total_rows`, `best_age['age_group']`), never typed by hand. If the data changed, the written conclusions would change with it, so the printed text can never contradict the printed tables.

A third helper, `get_metrics()`, is defined before the modelling objectives. It computes the confusion-matrix counts and all evaluation metrics for a set of predictions, so that both classifiers are evaluated by exactly the same code. It is described in Chapter 8.

4.4 Spark Configuration Choices

Two configuration settings are applied immediately after the session is obtained:

```
spark.sparkContext.setLogLevel("ERROR")
spark.conf.set("spark.sql.shuffle.partitions", "8")
```

- **`setLogLevel("ERROR")`** suppresses Spark's informational and warning messages so that the console shows only the program's own output, which keeps each objective readable.
- **`spark.sql.shuffle.partitions = 8`** reduces the number of partitions created after a shuffle (for example after `groupBy` or a join) from the default of 200 to 8. On a single machine with a few cores and a 45,000-row dataset, 200 tiny partitions would create unnecessary scheduling overhead. Eight partitions match the parallelism of a typical laptop. On a real cluster with much larger data, this value would be increased.

The program also uses **caching** deliberately. The base DataFrame `df` is cached because it is reused by almost every objective; without caching, Spark would re-read and re-parse the CSV file for every action. The training and test sets, the encoded feature DataFrames and the prediction DataFrames are cached for the same reason, and all of them are released with `unpersist()` at the end of the program.

Finally, the program fixes the random seed (`seed=42`) for both the train/test split and the Decision Tree. Fixed seeds make the results repeatable on the same machine and Spark configuration. Because `randomSplit` samples within each partition, the exact split can differ slightly if the number of input partitions changes, so the counts reported in this report correspond to the configuration described above.

Chapter 5: Data Loading, Quality and Exploratory Analysis (Objectives 1–5)

The first five objectives establish what the data contains and whether it can be trusted. The dataset loaded cleanly with 45,211 rows and 17 columns, had no nulls or duplicates, and showed a strong 88.30% to 11.70% class imbalance.

5.1 Objective 1: Load the Dataset and Inspect Its Structure

Problem statement. Load the bank marketing dataset and identify its number of records, columns, and data types.

Approach. The CSV file is read with Spark's DataFrame reader. Four options are set: `header` uses the first line as column names, `sep` sets the semicolon separator, `inferSchema` asks Spark to scan the data and detect integer columns automatically, and `mode=FAILFAST` makes Spark raise an error immediately if any row is malformed, rather than silently dropping or nulling it. The DataFrame is cached because nearly every later objective reuses it. `count()` is the first action and triggers the actual read.

```
df = (
    spark.read
    .option("header", True)
    .option("sep", ";")
    .option("inferSchema", True)
    .option("mode", "FAILFAST")
    .csv(path)
    .cache()
)

total_rows = df.count()
```

Output. Spark reported **45,211 rows and 17 columns**. The inferred schema is summarised below.

Table 5.1: Inferred schema of the dataset

Data type	Columns	Count
integer	age, balance, day, duration, campaign, pdays, previous	7
string	job, marital, education, default, housing, loan, contact, month, poutcome, y	10

The first five rows displayed by `df.show(5)` were:

```
+---+-----------+-------+---------+-------+-------+-------+----+-------+---
+-----+--------+--------+-----+--------+--------+---+
|age|job        |marital|education|default|balance|housing|loan|contact|day|
month|duration|campaign|pdays|previous|poutcome|y  |
+---+-----------+-------+---------+-------+-------+-------+----+-------+---
+-----+--------+--------+-----+--------+--------+---+
|58 |management  |married|tertiary |no     |2143   |yes    |no  |unknown|5  |
may  |261     |1       |-1   |0       |unknown |no |
|44 |technician  |single |secondary|no     |29     |yes    |no  |unknown|5  |
may  |151     |1       |-1   |0       |unknown |no |
|33 |entrepreneur|married|secondary|no     |2      |yes    |yes |unknown|5  |
may  |76      |1       |-1   |0       |unknown |no |
|47 |blue-collar |married|unknown  |no     |1506   |yes    |no  |unknown|5  |
may  |92      |1       |-1   |0       |unknown |no |
|33 |unknown     |single |unknown  |no     |1      |no     |no  |unknown|5  |
may  |198     |1       |-1   |0       |unknown |no |
+---+-----------+-------+---------+-------+-------+-------+----+-------+---
+-----+--------+--------+-----+--------+--------+---+
```

Interpretation. Schema inference correctly typed all seven numerical attributes as integers, so no manual casting was needed. Because the binary columns (default, housing, loan, y) contain the strings "yes" and "no", they were inferred as strings and must be converted before modelling. The sample rows already show the special values discussed in Chapter 3: *contact* and *poutcome* are "unknown" and *pdays* is −1 for clients never contacted before.

5.2 Objective 2: Null Values, Duplicates and "unknown" Values

Problem statement. Identify null values, exact duplicate records, and unknown categorical values.

Approach. Three checks are performed, each as a single distributed pass over the data:

- **Nulls.** For every column, `isNull()` is cast to an integer (1 for null, 0 otherwise) and summed. Using a list comprehension inside one `select` computes all 17 counts in one job.
- **Duplicates.** `dropDuplicates()` removes rows identical in every column; subtracting its count from the total gives the number of exact duplicates.
- **"unknown" values.** For every string column, `F.when(col == "unknown", 1).otherwise(0)` is summed.

```
null_counts = df.select([
    F.sum(F.col(c).isNull().cast("int")).alias(c)
    for c in df.columns
])

duplicate_count = total_rows - df.dropDuplicates().count()
```

Output. All 17 null counts were 0, and the number of exact duplicate rows was 0. The "unknown" counts were:

Table 5.2: Count of "unknown" values per categorical column

Column	"unknown" count	Share of 45,211 rows
poutcome	36,959	81.75%
contact	13,020	28.80%
education	1,857	4.11%
job	288	0.64%
marital, default, housing, loan, month, y	0	0.00%

Interpretation. The dataset is technically complete, but "unknown" plays the role of a missing value in four columns. The program keeps "unknown" as a legitimate category rather than deleting those rows, for two reasons. First, deleting rows with unknown *poutcome* would remove 81.75% of the data. Second, "unknown" often carries information: an unknown *poutcome* almost always means the client was never contacted before, which is itself a meaningful state. The program also notes that *pdays* = −1 is a code, not a real number of days.

5.3 Objective 3: Descriptive Statistics

Problem statement. Calculate descriptive statistics for customer age, balance, and numerical campaign attributes.

Approach. The DataFrame method `summary()` computes count, mean, standard deviation, minimum, quartiles and maximum for all seven numerical columns in one call. An additional `agg()` extracts the minimum, maximum and mean age and the mean balance so that they can be inserted into the written conclusion.

```
df.select(numeric_columns).summary(
    "count", "mean", "stddev",
    "min", "25%", "50%", "75%", "max"
).show(truncate=False)
```

Output.

Table 5.3: Descriptive statistics of numerical attributes (values rounded to two decimals)

Statistic	age	balance	day	duration	campaign	pdays	previous
count	45,211	45,211	45,211	45,211	45,211	45,211	45,211
mean	40.94	1,362.27	15.81	258.16	2.76	40.20	0.58
stddev	10.62	3,044.77	8.32	257.53	3.10	100.13	2.30
min	18	−8,019	1	0	1	−1	0
25%	33	72	8	103	1	−1	0
50% (median)	39	448	16	180	2	−1	0
75%	48	1,426	21	319	3	−1	0
max	95	102,127	31	4,918	63	871	275

The program's conclusion: customer ages range from 18 to 95 years with an average of 40.94 years, and the average yearly balance is 1,362.27 euros.

Interpretation.

- **Age** is fairly concentrated: half of the clients are between 33 and 48 years old.
- **Balance** is extremely skewed. The mean (1,362) is three times the median (448), the standard deviation (3,045) is more than twice the mean, and the range runs from an overdraft of −8,019 to 102,127 euros. A few very wealthy clients pull the mean upwards, so the median is the better description of a typical client.
- **Duration** has a median of 180 seconds (three minutes) but a maximum of 4,918 seconds (82 minutes).
- **Campaign** shows that most clients were called once to three times, but at least one client was called 63 times in the same campaign.
- **pdays** and **previous** are dominated by clients never contacted before: all three quartiles of *pdays* are −1 and all three quartiles of *previous* are 0. The mean of *pdays* (40.20) is therefore not meaningful as an average number of days.

5.4 Objective 4: Target Distribution and Class Imbalance

Problem statement. Calculate subscriber and non-subscriber counts and identify class imbalance.

Approach. The data is grouped by *y* and each count is converted to a percentage of the total. The objective also creates **`analysis_df`**, a copy of the data with a numeric `label` column (1.0 for yes, 0.0 for no) that every later objective uses. The filter `isin("yes", "no")` guarantees that only valid target values are kept.

```
analysis_df = (
    df.filter(F.col("y").isin("yes", "no"))
    .withColumn(
        "label",
        F.when(F.col("y") == "yes", 1.0).otherwise(0.0)
```

```
    )
)
```

Output.

Table 5.4: Target class distribution

y	label	Count	Percentage
no	0.0	39,922	88.30%
yes	1.0	5,289	11.70%
Total		**45,211**	**100.00%**

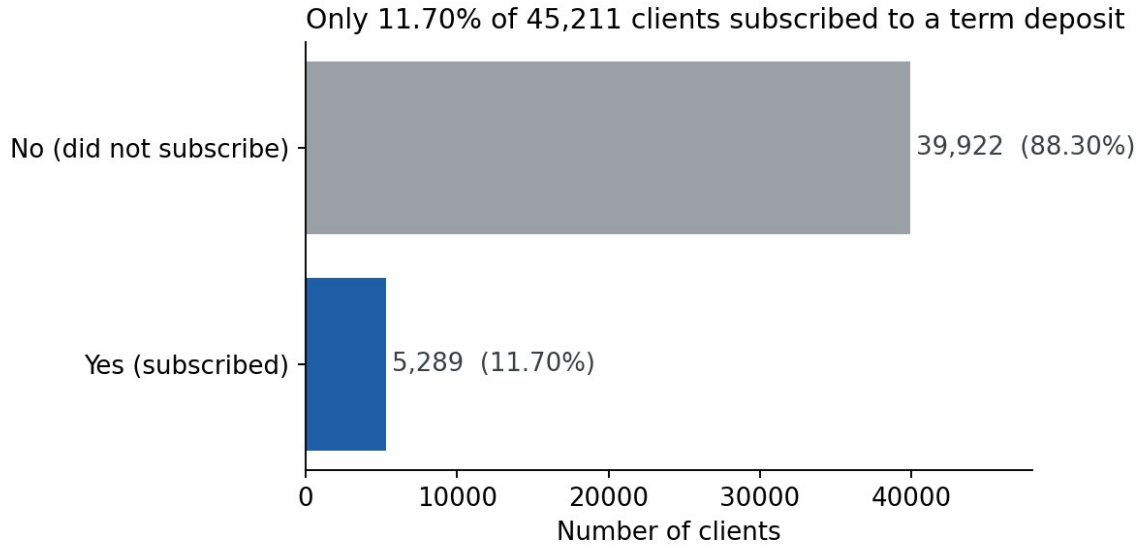


Figure 5.1: Distribution of the target variable. Non-subscribers outnumber subscribers by about 7.5 to 1.

Interpretation. The dataset is clearly imbalanced. This has two consequences that shape the rest of the project. First, any subscription rate for a segment should be compared with the overall rate of 11.70%: a segment at 15% is above average, not "low". Second, a classifier that always predicts "no" would be about 88% accurate while being useless to the bank, so accuracy cannot be the main measure of model quality (Objective 15).

A useful property of the 0/1 label is that the **average of the label within any group equals the subscription rate of that group**. This is why later objectives compute the rate simply as `AVG(label) * 100`.

5.5 Objective 5: Occupation Counts using RDD Operations

Problem statement. Use RDD map and reduceByKey operations to count customers belonging to each occupation.

Approach. This objective deliberately uses the low-level RDD API to demonstrate the MapReduce pattern from Unit III. The *job* column is converted to an RDD of `Row` objects. A `map` transformation turns every row into a key–value pair `(job, 1)`, and `reduceByKey` adds up the 1s for each job. Finally `toDF()` converts the result back to a DataFrame for sorting and display, demonstrating RDD-to-DataFrame conversion from Unit IV.

```
job_counts_rdd = (
    df.select("job").rdd
```

```
    .map(lambda row: (row["job"], 1))
    .reduceByKey(lambda a, b: a + b)
)

job_counts_df = job_counts_rdd.toDF(
    ["job", "customer_count"]
).orderBy(
    F.desc("customer_count"), "job"
)
```

The execution has two stages. In the first stage, each partition runs `map` and a **local combine** that pre-sums counts inside the partition. In the second stage, partial counts for the same job are **shuffled** to the same partition and added together. Because of the local combine, at most twelve pairs per partition cross the shuffle boundary instead of one pair per customer.

Output.

Table 5.5: Customers per occupation (RDD output)

Rank	Job	Customers	Share
1	blue-collar	9,732	21.53%
2	management	9,458	20.92%
3	technician	7,597	16.80%
4	admin.	5,171	11.44%
5	services	4,154	9.19%
6	retired	2,264	5.01%
7	self-employed	1,579	3.49%
8	entrepreneur	1,487	3.29%
9	unemployed	1,303	2.88%
10	housemaid	1,240	2.74%
11	student	938	2.07%
12	unknown	288	0.64%

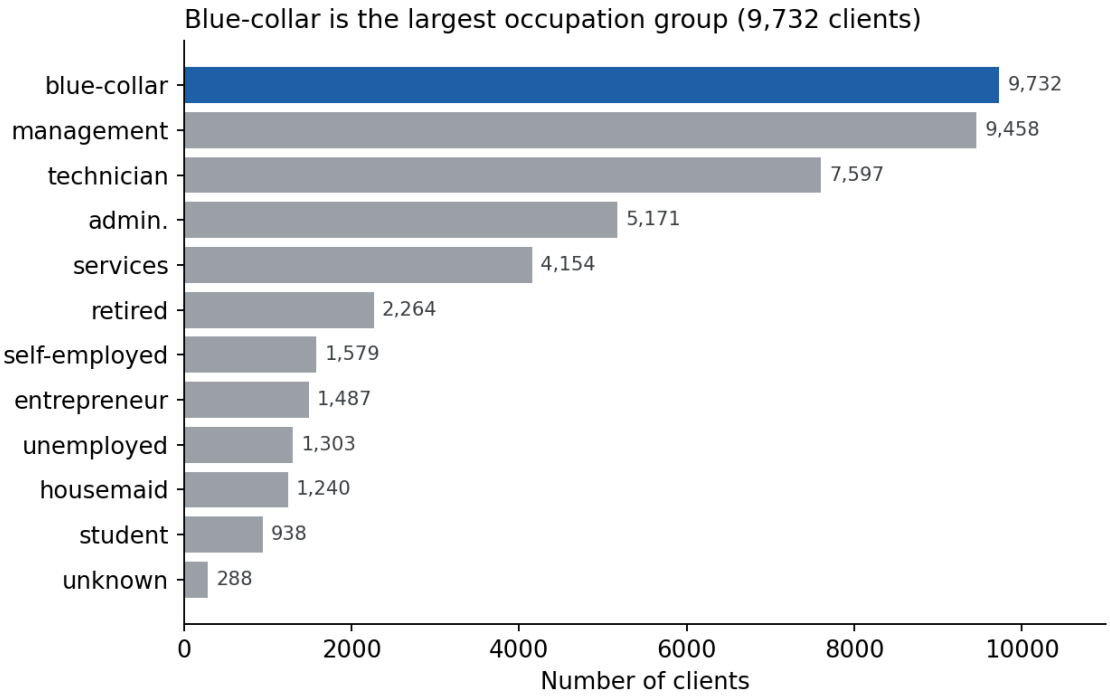


Figure 5.2: Customer count by occupation, computed with RDD `map` and `reduceByKey`.

Interpretation. The twelve counts add up to exactly 45,211, confirming that every record was counted once. Blue-collar (9,732) and management (9,458) together make up 42.45% of all clients, and the top three occupations account for almost 60%. Students (938) and the unknown category (288) are small groups, so their subscription rates in later objectives rest on fewer observations and should be read with more caution. The same counts could be produced in one line with `df.groupBy("job").count()`, which Spark would optimise automatically; the RDD version is used here to show the underlying map and reduce mechanics.

Chapter 6: Segment Analysis with DataFrames and Spark SQL (Objectives 6–10)

Objectives 6 to 10 compare subscription rates across client segments, using both the DataFrame API and Spark SQL. The strongest signals are a successful previous campaign (64.73%), age 60 and above (33.63%) and being a student (28.68%); all segment rates should be read against the overall rate of 11.70%.

6.1 Objective 6: Subscription Rate by Age Group

Problem statement. Group customers by age and compare their term deposit subscription rates.

Approach. A new column `age_group` is derived with a chain of `F.when(...).otherwise(...)` expressions, the DataFrame equivalent of an SQL `CASE WHEN`. The conditions are checked in order, so each client falls into exactly one band. For each band the program counts customers, sums the label to get the number of subscribers, and averages the label to get the rate.

```
age_df = analysis_df.withColumn(
    "age_group",
    F.when(F.col("age") < 30, "Below 30")
    .when(F.col("age") < 40, "30-39")
    .when(F.col("age") < 50, "40-49")
    .when(F.col("age") < 60, "50-59")
    .otherwise("60 and above")
)

age_summary = age_df.groupBy("age_group").agg(
    F.count("*").alias("customers"),
    F.sum("label").cast("long").alias("subscribers"),
    (F.avg("label") * 100).alias("subscription_rate_pct")
).orderBy(F.desc("subscription_rate_pct"))
```

Output.

Table 6.1: Subscription rate by age group

Age group	Customers	Subscribers	Subscription rate
60 and above	1,784	600	33.63%
Below 30	5,273	928	17.60%
30–39	18,089	1,913	10.58%
50–59	8,410	785	9.33%
40–49	11,655	1,063	9.12%

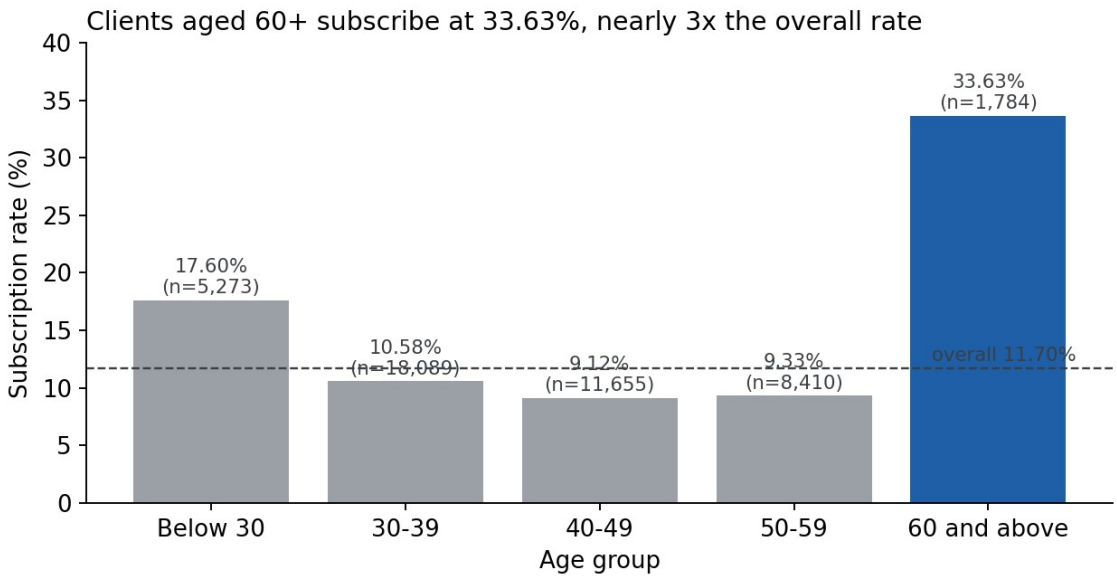


Figure 6.1: Subscription rate by age group. The dashed line marks the overall rate of 11.70%.

Interpretation. The relationship between age and subscription is **U-shaped**. Clients aged 60 and above subscribe at 33.63%, nearly three times the overall rate, and clients below 30 subscribe at 17.60%. The middle-aged bands (30–59), which contain 84% of all clients, are all below the overall average. A plausible explanation is that older clients, many of them retired, have savings to place and prefer low-risk products, while younger clients include students who may be offered introductory products. The 60+ group is small (1,784 clients), so although the rate is very high, it contributes only 600 subscribers, fewer than the 30–39 group (1,913). This shows the difference between targeting by **rate** and by **volume**.

6.2 Objective 7: Occupation and Education using Spark SQL

Problem statement. Use Spark SQL to compare subscription rates by occupation and education level.

Approach. `analysis_df` is registered as a temporary view named `bank` with `createOrReplaceTempView`. Two SQL queries are then run with `spark.sql()`. Each query groups by one attribute, counts customers and computes the rate as `AVG(label) * 100`, sorted from highest to lowest. The temporary view exists only for the lifetime of the Spark session and does not copy the data.

```
SELECT
    job,
    COUNT(*) AS customers,
    AVG(label) * 100 AS subscription_rate_pct
FROM bank
GROUP BY job
ORDER BY subscription_rate_pct DESC
```

Output (occupation). The rates by occupation are shown in Figure 6.2; the full numbers are in Appendix A.

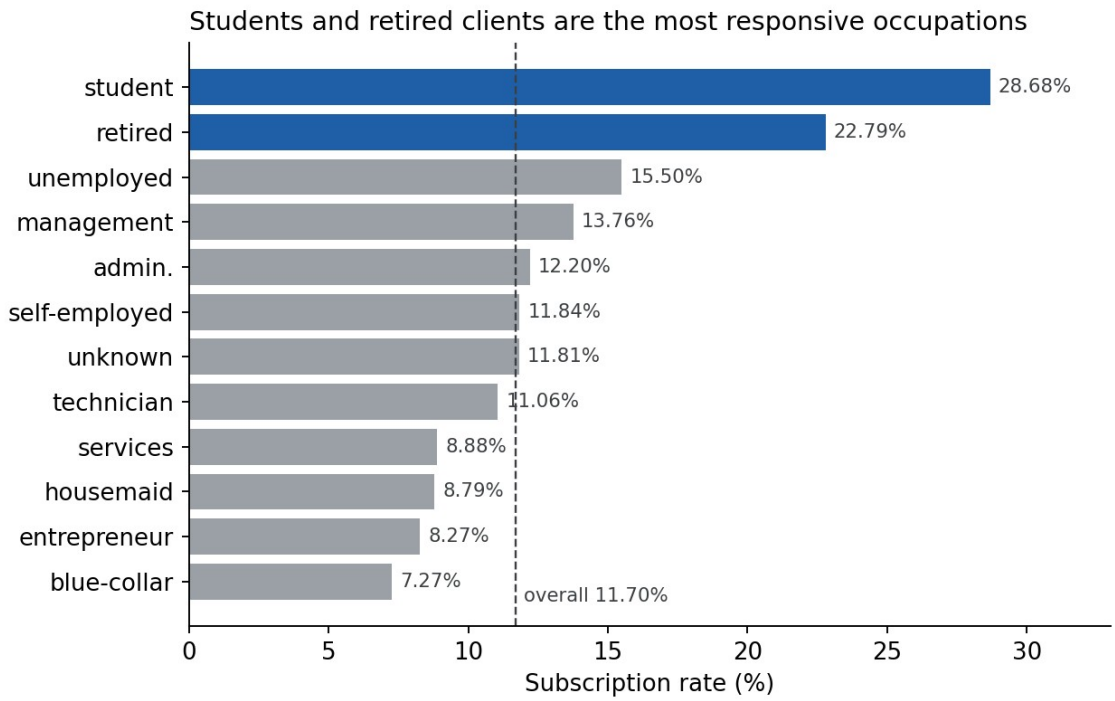


Figure 6.2: Subscription rate by occupation (Spark SQL). Students and retired clients are highlighted; the dashed line marks the overall rate.

Output (education).

Table 6.2: Subscription rate by education level

Education	Customers	Subscription rate
tertiary	13,301	15.01%
unknown	1,857	13.57%
secondary	23,202	10.56%
primary	6,851	8.63%

Interpretation. Students (28.68%) and **retired clients (22.79%)** are by far the most responsive occupations, consistent with the U-shaped age pattern from Objective 6. Unemployed clients (15.50%) and management (13.76%) are also above average. At the other end, **blue-collar workers have the lowest rate (7.27%)**, even though they are the largest group (Objective 5). Entrepreneurs (8.27%), housemaids (8.79%) and services (8.88%) are also well below average.

For education, the rate rises steadily with the level of education, from 8.63% (primary) to 10.56% (secondary) to 15.01% (tertiary). Education and occupation are related: management roles usually require tertiary education, while blue-collar roles often require less. The two findings therefore partly describe the same underlying pattern, and should not be treated as independent effects.

6.3 Objective 8: Housing and Personal Loans

Problem statement. Compare subscription rates for different combinations of housing loans and personal loans.

Approach. The DataFrame is grouped by two columns at once, `housing` and `loan`, which produces one row for each of the four combinations. For each combination the program also computes the average balance, to see whether loan status is linked to the client's financial position.

```
loan_summary = analysis_df.groupBy("housing", "loan").agg(
    F.count("*").alias("customers"),
    F.round(F.avg("balance"), 2).alias("average_balance"),
    (F.avg("label") * 100).alias("subscription_rate_pct")
).orderBy(F.desc("subscription_rate_pct"))
```

Output.

Table 6.3: Subscription rate by housing and personal loan status

Housing loan	Personal loan	Customers	Average balance (€)	Subscription rate
no	no	17,204	1,737.61	18.22%
yes	no	20,763	1,256.41	8.04%
no	yes	2,877	752.71	7.61%
yes	yes	4,367	788.54	6.07%

Interpretation. Clients with **no loans at all** subscribe at 18.22%, more than double any other group, and they also have the highest average balance (1,737.61 euros). Having either kind of loan cuts the rate to about 8%, and having both cuts it to 6.07%. Clients with a personal loan have less than half the average balance of loan-free clients. A client who is repaying debt has less free money to lock into a term deposit, which explains the pattern. As the program's conclusion notes, this is an **association, not proof of causation**: loan-free clients also tend to be older or retired, so age and loan status overlap.

6.4 Objective 9: Contact Method and Campaign Month

Problem statement. Compare historical subscription rates across contact methods and campaign months.

Approach. Two separate `groupBy` aggregations are run, one on `contact` and one on `month`. Customer counts are reported next to every rate so that small groups can be identified.

Output (contact method).

Table 6.4: Subscription rate by contact method

Contact method	Customers	Subscription rate
cellular	29,285	14.92%
telephone	2,906	13.42%
unknown	13,020	4.07%

Output (campaign month). Figure 6.3 plots the rate for each month together with the number of clients contacted in that month.

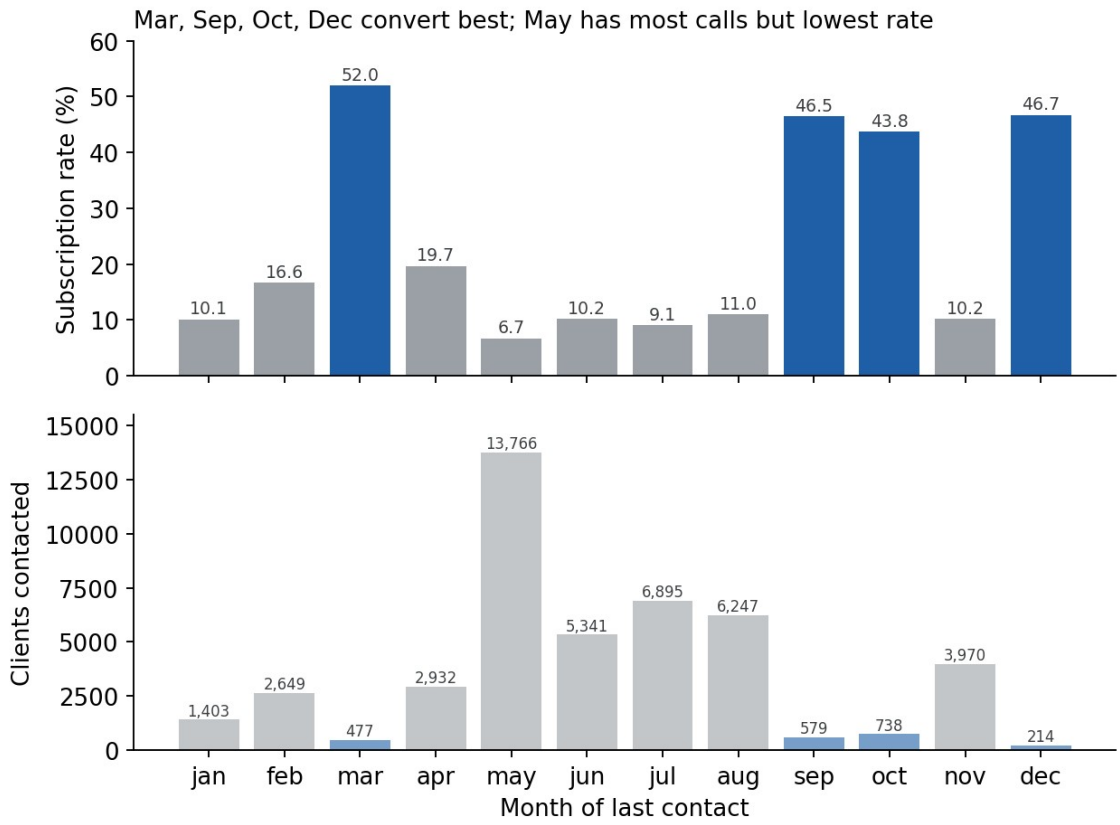


Figure 6.3: Subscription rate (top) and number of clients contacted (bottom) by month of last contact.

Interpretation. Cellular (14.92%) and landline telephone (13.42%) perform similarly, but clients whose contact type is **unknown subscribe at only 4.07%**. Most of the unknown-contact records come from the early part of the data (the first rows in Objective 1 were all from May with unknown contact), when the bank apparently did not record the channel; the low rate is likely to reflect that early period rather than the channel itself.

The monthly pattern is striking. March (51.99%), December (46.73%), September (46.46%) and October (43.77%) have subscription rates around four times the average, but they are also the **four months with the fewest contacts** (between 214 and 738 clients each). May, the busiest month with 13,766 contacts (30% of all records), has the **lowest** rate at 6.72%. This inverse relationship suggests that in low-volume months the bank called a smaller, more carefully selected group of clients, perhaps those who had shown interest earlier, while in May it called clients on a large scale. The month therefore reflects **campaign strategy and client selection** as much as seasonality. Rates computed from a few hundred records are also less stable than those from thousands.

6.5 Objective 10: Previous Campaign Outcome using an SQL LEFT JOIN

Problem statement. Use an SQL LEFT JOIN to attach descriptions of previous campaign outcomes and compare current subscription rates.

Approach. A small lookup DataFrame is created with `spark.createDataFrame()`, mapping each outcome code to a readable description, and registered as the view `outcome_lookup`. The main `bank` view is then joined to it with a **LEFT JOIN** on `poutcome = outcome_code`. A LEFT JOIN keeps every row of the left table (all 45,211 clients) even if no matching description existed, so no client can be lost through the join. The result is grouped by outcome to compute the current subscription rate.

```
SELECT
    b.poutcome,
```

```
    o.outcome_description,
    COUNT(*) AS customers,
    AVG(b.label) * 100 AS subscription_rate_pct
FROM bank b
LEFT JOIN outcome_lookup o
    ON b.poutcome = o.outcome_code
GROUP BY b.poutcome, o.outcome_description
ORDER BY subscription_rate_pct DESC
```

Because the lookup table has only four rows, Spark automatically uses a **broadcast hash join**: it sends a copy of the small table to every executor, so the large table does not need to be shuffled.

Output.

Table 6.5: Subscription rate by previous campaign outcome (LEFT JOIN)

poutcome	Description	Customers	Subscription rate
success	Previous campaign succeeded	1,511	64.73%
other	Other previous outcome	1,840	16.68%
failure	Previous campaign failed	4,901	12.61%
unknown	Previous outcome unknown	36,959	9.16%

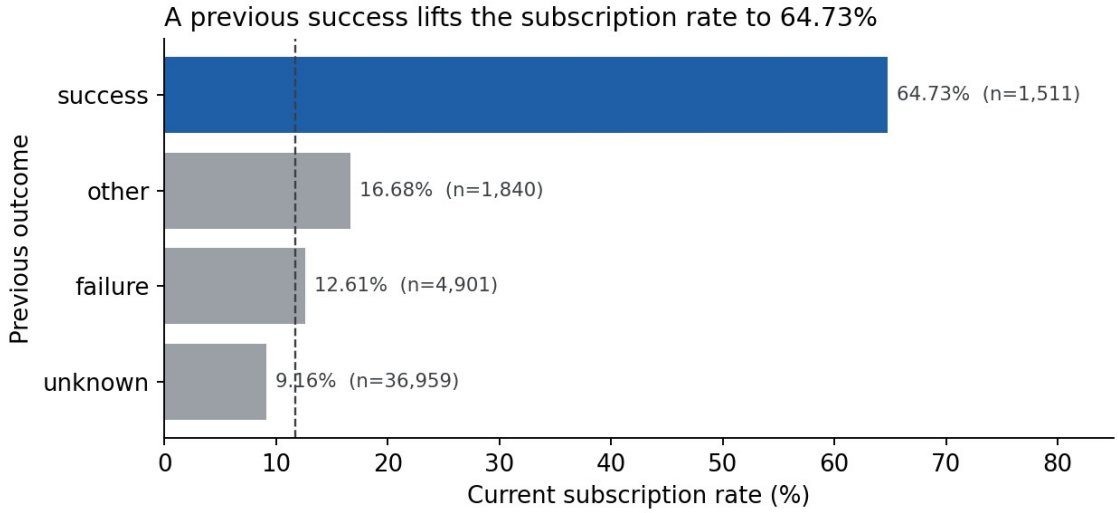


Figure 6.4: Current subscription rate by outcome of the previous campaign.

Interpretation. Every row received a description, and the four customer counts add up to 45,211, confirming the LEFT JOIN kept all records. The previous outcome is the **strongest single predictor** found in the exploratory analysis. Clients who responded positively to an earlier campaign subscribed again at **64.73%**, about 5.5 times the overall rate. Even clients whose previous campaign failed (12.61%) or ended in another outcome (16.68%) did slightly better than clients with no history (9.16%), suggesting that any previous engagement indicates some familiarity with the bank's products. The practical message for the bank is clear: past responders are the most valuable list to call first.

Chapter 7: Feature Engineering and the ML Pipeline (Objectives 11–12)

Objectives 11 and 12 turn the cleaned data into model-ready input: 15 leak-free attributes, a reproducible 36,194 / 9,017 train–test split, and a 59-dimensional feature vector produced by a pipeline fitted on training data only.

7.1 Objective 11: Feature Selection and Train–Test Split

Problem statement. Select prediction features and split the data into approximately 80% training and 20% testing.

Approach. The most important design decision in this objective is **which attributes the model is allowed to see**. A predictive model is only useful if its inputs are available at the moment the prediction is needed, which here is **before the bank makes the call**. The program applies three rules, documented in comments in the code:

```
# Duration is excluded because it is known after the call.
# The target y must not be included as an input feature.
# Contact date, method and attempt count are assumed known
# at prediction time.
```

- **duration is excluded.** The length of a call is only known once the call has ended, at which point the outcome is also known. Including it would give the model an unrealistically high score that it could never achieve in practice. This is a form of **target leakage**.
- **y is excluded** as an input, because it is the answer the model is trying to predict.
- **contact, day, month and campaign are included**, on the assumption that the bank decides the channel, the calendar date and the contact attempt number before dialling.

Table 7.1: Input features selected for modelling

Type	Count	Features
Categorical	9	job, marital, education, default, housing, loan, contact, month, poutcome
Numerical	6	age, balance, day, campaign, pdays, previous
Excluded	2	duration (post-call leakage), y (target)
Target	1	label (0.0 = no, 1.0 = yes)

Two defensive cleaning steps are applied: `na.fill("unknown")` replaces any null categorical value with “unknown”, and `na.drop` removes rows with missing numerical values. Objective 2 showed there are no nulls, so neither step removes data here, but they make the pipeline safe if the code is reused on a different file.

The data is then split with `randomSplit([0.8, 0.2], seed=42)`. Spark assigns each row to the training or test set independently with the given probabilities, so the split is approximately, not exactly, 80/20. Both sets are cached because they are used several times.

```
train_df, test_df = model_df.randomSplit(
    [0.8, 0.2],
    seed=42
)
```

Output.

Table 7.2: Train/test split and label distribution

Set	Records	Share of total	label = 0 (no)	label = 1 (yes)	Positive rate
Training	36,194	80.06%	31,910	4,284	11.84%
Testing	9,017	19.94%	8,012	1,005	11.15%
Total	**45,211**	**100.00%**	**39,922**	**5,289**	**11.70%**

Interpretation. The two sets add up to the full 45,211 records, so no data was lost. The split is almost exactly 80/20, and the proportion of subscribers is similar in both sets (11.84% and 11.15%), so the test set is representative of the training data. Because the split is random rather than stratified, the positive rates are not identical; a stratified split (sampling each class separately) would make them equal. As the program's conclusion states, this random split is a **coursework holdout experiment**, not a time-based evaluation of future campaigns: the bank would really apply the model to calls in the future, which a split on date would simulate more faithfully.

7.2 Objective 12: Encoding and Feature Assembly with a Spark ML Pipeline

Problem statement. Encode categorical variables and combine all inputs into a feature vector using a Spark ML pipeline.

Approach. Machine learning algorithms in MLlib operate on numbers, and they require all inputs in a single vector column. The program builds a **Pipeline** with eleven stages:

1. **Nine StringIndexers**, one per categorical column. Each converts category strings into numeric indices ordered by frequency (the most frequent category gets index 0). `handleInvalid="keep"` reserves an extra index for categories that appear in new data but were not seen in training, so the model does not crash on unexpected values.
2. **One OneHotEncoder** handling all nine index columns. It converts each index into a sparse binary vector, so that the model does not mistakenly treat, for example, "month index 5" as five times "month index 1". `handleInvalid="keep"` again provides a slot for unseen categories.
3. **One VectorAssembler**, which concatenates the six numerical columns and the nine encoded vectors into the single column `features`.

```
indexers = [
    StringIndexer(
        inputCol=c,
        outputCol=c + "_index",
        handleInvalid="keep"
    )
    for c in categorical_features
]

encoder = OneHotEncoder(
    inputCols=[c + "_index" for c in categorical_features],
    outputCols=[c + "_encoded" for c in categorical_features],
    handleInvalid="keep"
)

assembler = VectorAssembler(
    inputCols=(
        numeric_features
        + [c + "_encoded" for c in categorical_features]
    ),
    outputCol="features"
)

preprocessing_pipeline = Pipeline(
    stages=indexers + [encoder, assembler]
```

```
)

# Fit preprocessing only on training data.
preprocessing_model = preprocessing_pipeline.fit(train_df)
```

The crucial line is `preprocessing_pipeline.fit(train_df)`. The pipeline **learns** the list of categories and their indices from the **training set only**, and the fitted `preprocessing_model` is then applied, unchanged, to both sets. If the pipeline were fitted on the full data, information from the test set (for example, which categories exist and how frequent they are) would leak into the preprocessing, and the test results would be slightly optimistic. Fitting on training data only mirrors real use, where the model is built once and then applied to clients it has never seen.

Output. The first five encoded training rows were:

```
+--------------------------------------------------------------------------------
----------------------------+-----+
|features
|label|
+--------------------------------------------------------------------------------
----------------------------+-----+
|(59,[0,1,2,3,4,9,21,25,28,32,34,37,42,54],[36.0,227.0,9.0,1.0,-
1.0,1.0,1.0,1.0,1.0,1.0,1.0,1.0,1.0,1.0]) |0.0  |
|(59,[0,1,2,3,4,9,21,25,28,32,34,37,42,54],[46.0,509.0,17.0,1.0,-
1.0,1.0,1.0,1.0,1.0,1.0,1.0,1.0,1.0,1.0])|0.0  |
|(59,[0,1,2,3,4,9,21,25,28,32,34,38,44,54],[57.0,987.0,16.0,2.0,-
1.0,1.0,1.0,1.0,1.0,1.0,1.0,1.0,1.0,1.0])|0.0  |
|(59,[0,1,2,3,4,9,21,25,28,32,34,38,41,54],[57.0,1.0,15.0,10.0,-
1.0,1.0,1.0,1.0,1.0,1.0,1.0,1.0,1.0,1.0]) |0.0  |
|(59,[0,1,2,3,4,9,21,25,28,32,35,37,42,54],[46.0,505.0,18.0,4.0,-
1.0,1.0,1.0,1.0,1.0,1.0,1.0,1.0,1.0,1.0])|0.0  |
+--------------------------------------------------------------------------------
----------------------------+-----+
```

Each vector is printed in **sparse format** as `(size, [indices], [values])`. The first row, for example, has size 59 and only 14 non-zero entries: positions 0 to 4 hold age 36, balance 227, day 9, campaign 1 and pdays −1 (previous is 0, so position 5 is omitted), and each of the nine categorical attributes contributes exactly one 1.0. Storing only the 14 non-zero values instead of all 59 saves memory, which matters at scale.

Table 7.3: Layout of the 59-dimensional feature vector (determined by inspecting the fitted StringIndexer labels)

Positions	Source	Slots	Notes
0–5	age, balance, day, campaign, pdays, previous	6	Raw numerical values
6–18	job	13	12 categories + 1 unseen slot
19–22	marital	4	3 categories + 1
23–27	education	5	4 categories + 1
28–30	default	3	2 categories + 1
31–33	housing	3	2 categories + 1
34–36	loan	3	2 categories + 1
37–40	contact	4	3 categories + 1
41–53	month	13	12 categories + 1

Positions	Source	Slots	Notes
54–58	poutcome	5	4 categories + 1
Total		**59**	6 numerical + 53 encoded

Interpretation. The nine categorical attributes have 44 categories in total. With `handleInvalid="keep"` on both the indexer and the encoder, each attribute receives one slot per category plus one slot for unseen values (the encoder's default `dropLast=True` removes one slot from the k + 2 index values, leaving k + 1). This gives 44 + 9 = 53 encoded positions, plus 6 numerical positions, for a total of 59, exactly as the program reported. The pipeline object can be saved and reused, so the same transformation can be applied to next month's call list with a single `transform()` call.

Chapter 8: Model Training and Evaluation (Objectives 13–15)

Both classifiers reach about 89.4–89.6% accuracy, barely above the 88.85% obtained by always predicting "no". The Decision Tree is the more useful model for finding subscribers: it recalls 16.92% of them against 7.96% for Logistic Regression, while Logistic Regression ranks clients better overall (ROC-AUC 0.7788 vs 0.6821).

8.1 The Common Evaluation Helper

To guarantee a fair comparison, both models are evaluated by the same function, `get_metrics()`. Three built-in MLlib evaluators compute accuracy, ROC-AUC and PR-AUC. The confusion-matrix counts are computed directly by grouping the predictions on `label` and `prediction`, and precision, recall and F1 for the **positive class (yes = 1)** are derived from those counts.

```
def get_metrics(name, predictions):
    counts = {
        (int(row["label"]), int(row["prediction"])): row["count"]
        for row in predictions.groupBy(
            "label", "prediction"
        ).count().collect()
    }

    tn = counts.get((0, 0), 0)
    fp = counts.get((0, 1), 0)
    fn = counts.get((1, 0), 0)
    tp = counts.get((1, 1), 0)

    precision = tp / (tp + fp) if tp + fp else 0.0
    recall = tp / (tp + fn) if tp + fn else 0.0
    ...
```

Three details make the helper robust. The `counts.get(..., 0)` calls return zero for any cell that does not appear (for example, if a model never predicts "yes"). The conditional expressions avoid division by zero. And the ROC and PR evaluators use the `probability` column rather than the hard 0/1 `prediction`, so that they measure how well the model **ranks** clients across all thresholds. Note that the precision, recall and F1 reported here are computed for the subscriber class only, which is the class the bank cares about; MLlib's default multiclass metrics would instead average over both classes and hide weak performance on subscribers.

8.2 Objective 13: Logistic Regression

Problem statement. Train a Logistic Regression classifier to predict term deposit subscriptions.

Approach. A `LogisticRegression` estimator is fitted on `train_ready` and applied to `test_ready`.

Table 8.1: Hyperparameters of the two classifiers

Model	Parameter	Value	Meaning
Logistic Regression	maxIter	100	Maximum optimisation iterations
Logistic Regression	regParam	0.1	Strength of regularisation
Logistic Regression	standardization	True	Features scaled internally before fitting
Logistic Regression	threshold	0.5 (default)	P(yes) above which a client is predicted "yes"

Model	Parameter	Value	Meaning
Decision Tree	maxDepth	5	Maximum number of levels of splits
Decision Tree	impurity	gini (default)	Split quality measure
Decision Tree	seed	42	Reproducibility

Standardisation matters because *balance* ranges into the tens of thousands while one-hot features are 0 or 1. Without scaling, the regularisation penalty would affect features very unequally. Spark standardises internally and reports coefficients on the original scale.

```
lr = LogisticRegression(
    featuresCol="features",
    labelCol="label",
    maxIter=100,
    regParam=0.1,
    standardization=True
)

lr_model = lr.fit(train_ready)
lr_predictions = lr_model.transform(test_ready).cache()
```

Output. The first ten test predictions were:

```
+-----+----------+----------------------------------------+
|label|prediction|probability                             |
+-----+----------+----------------------------------------+
|0.0  |0.0       |[0.9022997393568246,0.09770026064317539] |
|0.0  |0.0       |[0.8935937855693681,0.10640621443063192] |
|1.0  |0.0       |[0.5414673797810298,0.45853262021897023] |
|0.0  |0.0       |[0.9082736716042269,0.0917263283957731]  |
|0.0  |0.0       |[0.932657870997658,0.06734212900234204]  |
|0.0  |0.0       |[0.9427141591162717,0.057285840883728345]|
|1.0  |0.0       |[0.7820050856334486,0.21799491436655138] |
|0.0  |0.0       |[0.8739525552586637,0.12604744474133633] |
|0.0  |0.0       |[0.8556439960791734,0.1443560039208266]  |
|0.0  |0.0       |[0.8360430433043861,0.16395695669561394] |
+-----+----------+----------------------------------------+
only showing top 10 rows
```

The program reported **89.36% test accuracy** and **7.96% subscriber recall**, correctly identifying **80** of the 1,005 actual subscribers in the test set.

Interpretation. The probability vector shows [P(no), P(yes)] and the two values always sum to 1. Two of the ten sample rows are actual subscribers, and both were missed. The third row received P(yes) = 0.459, a high score compared with the other rows, but because it is below the 0.5 threshold it was predicted “no”; the seventh row received P(yes) = 0.218. This illustrates the main weakness of the model at its default threshold. Because subscribers are rare, very few clients ever reach P(yes) > 0.5, so the model predicts “yes” only 114 times out of 9,017 and misses most subscribers. The relatively strong regularisation (`regParam=0.1`) shrinks the coefficients and pulls probabilities towards the base rate, which makes this effect stronger.

8.3 Objective 14: Decision Tree

Problem statement. Train a Decision Tree classifier using the same training and testing data.

Approach. A `DecisionTreeClassifier` with maximum depth 5 is trained on exactly the same `train_ready` DataFrame and evaluated on the same `test_ready`. Decision trees do not need feature scaling, because each split compares one feature with a threshold.

```
dt = DecisionTreeClassifier(
    featuresCol="features",
    labelCol="label",
    maxDepth=5,
    seed=42
)

dt_model = dt.fit(train_ready)
dt_predictions = dt_model.transform(test_ready).cache()
```

Output. The first ten test predictions were:

```
+-----+----------+----------------------------------------+
|label|prediction|probability                             |
+-----+----------+----------------------------------------+
|0.0  |0.0       |[0.9595676927530638,0.040432307246936215]|
|0.0  |0.0       |[0.8836397290816541,0.11636027091834589] |
|1.0  |1.0       |[0.4351145038167939,0.5648854961832062]  |
|0.0  |0.0       |[0.8836397290816541,0.11636027091834589] |
|0.0  |0.0       |[0.9595676927530638,0.040432307246936215]|
|0.0  |0.0       |[0.9595676927530638,0.040432307246936215]|
|1.0  |0.0       |[0.8836397290816541,0.11636027091834589] |
|0.0  |0.0       |[0.8836397290816541,0.11636027091834589] |
|0.0  |0.0       |[0.8836397290816541,0.11636027091834589] |
|0.0  |0.0       |[0.8836397290816541,0.11636027091834589] |
+-----+----------+----------------------------------------+
only showing top 10 rows
```

The program reported **89.61% test accuracy** and **16.92% subscriber recall**, correctly identifying **170** of the 1,005 actual subscribers.

Interpretation. The subscriber in the third row, whom Logistic Regression missed, is correctly predicted by the Decision Tree with P(yes) = 0.565. The subscriber in the seventh row is still missed, because the tree places that client in a leaf with P(yes) = 0.116. Only three distinct probabilities (0.040, 0.116 and 0.565) appear in the ten rows, because every client that reaches the same leaf receives that leaf's class proportion as its probability. A tree of depth 5 has at most 32 leaves, so it can output only a small number of distinct probabilities. This coarse scoring explains why its ROC-AUC is lower than Logistic Regression's, even though it finds more subscribers.

8.4 Objective 15: Model Comparison and Baseline

Problem statement. Compare both models using accuracy, ROC-AUC, PR-AUC, subscriber precision, recall, F1, and confusion matrices.

Approach. The metric dictionaries returned by `get_metrics()` are converted into a Spark DataFrame and printed side by side, rounded to four decimals. For each model, the confusion matrix is displayed as a **pivot table**, with actual labels as rows and predicted labels as columns, using `groupBy("label").pivot("prediction", [0.0, 1.0])`. Listing the pivot values explicitly guarantees both columns appear even if a model never predicts one class. Finally, the **majority-class baseline** is computed as the share of "no" labels in the test set.

```
predictions.groupBy("label").pivot(
    "prediction", [0.0, 1.0]
).count().fillna(0).orderBy("label").show()

baseline_accuracy = (
    test_df.filter(F.col("label") == 0.0).count()
    / test_count
)
```

Output.

Table 8.2: Model comparison on the test set (9,048 records)

Metric	Logistic Regression	Decision Tree	Better
Accuracy	0.8950	0.8970	Decision Tree (slightly)
ROC-AUC	0.7725	0.6887	Logistic Regression
PR-AUC	0.3879	0.4016	Decision Tree
Precision (yes)	0.6879	0.6023	Logistic Regression
Recall (yes)	0.0967	0.2084	Decision Tree
F1 (yes)	0.1696	0.3096	Decision Tree
Baseline accuracy (always “no”)	0.8891	0.8891	–

Table 8.3: Confusion matrix counts

Model	TN	FP	FN	TP	Predicted “yes” (TP + FP)
Logistic Regression	8,001	44	906	97	141
Decision Tree	7,907	138	794	209	347

Decision Tree finds 209 of 1,003 subscribers; Logistic Regression finds 97

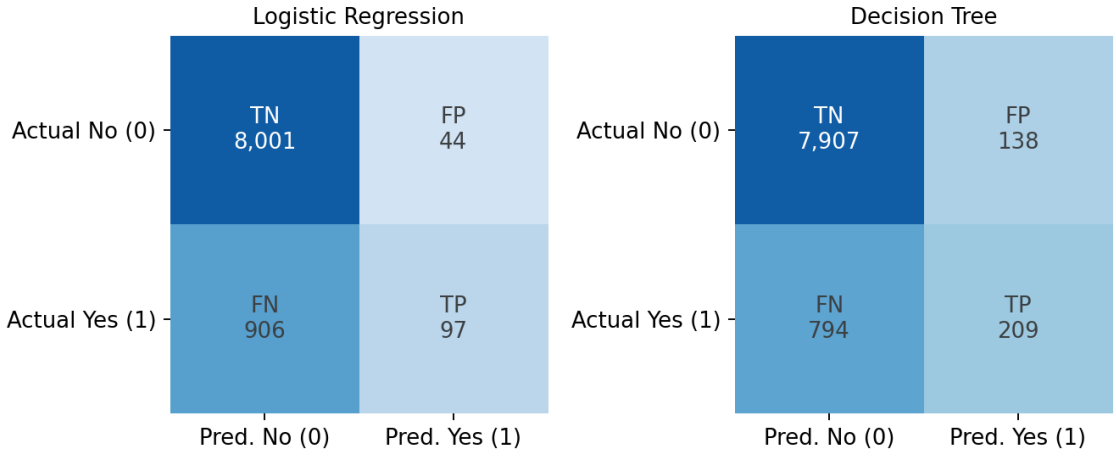


Figure 8.1: Confusion matrices of Logistic Regression and Decision Tree. Rows are actual labels and columns are predicted labels.

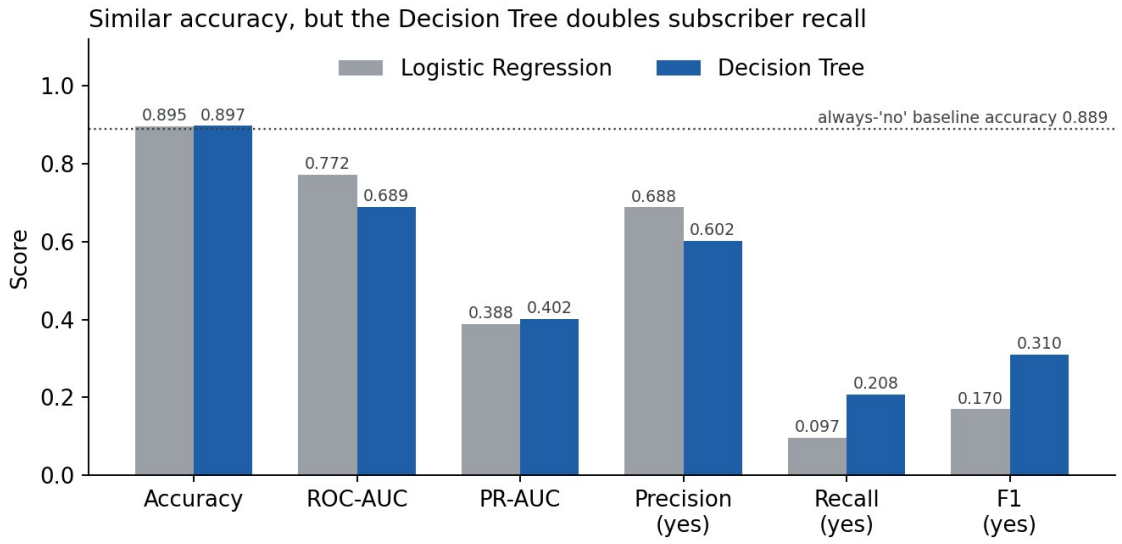


Figure 8.2: Comparison of evaluation metrics for both classifiers. The dotted line marks the 88.91% accuracy of always predicting “no”.

The metrics can be verified by hand from the confusion matrices. For the Decision Tree:

- Accuracy = (7,907 + 209) / 9,048 = 8,116 / 9,048 = 0.8970
- Precision = 209 / (209 + 138) = 209 / 347 = 0.6023
- Recall = 209 / (209 + 794) = 209 / 1,003 = 0.2084
- F1 = 2 × 0.6023 × 0.2084 / (0.6023 + 0.2084) = 0.3096

For Logistic Regression: accuracy = 8,098 / 9,048 = 0.8950, precision = 97 / 141 = 0.6879, recall = 97 / 1,003 = 0.0967 and F1 = 0.1696, all matching the program output.

Interpretation.

Accuracy is not a useful guide here. Always predicting “no” gives 8,045 / 9,048 = 88.91% accuracy. Logistic Regression improves on this by only 0.59 percentage points and the Decision Tree by 0.79. Judged by accuracy alone, both models look excellent, yet one of them misses more than 90% of subscribers.

The Decision Tree is better at finding subscribers. It identifies 209 of 1,003 subscribers, more than double the 97 found by Logistic Regression, giving almost twice the F1-score (0.3096 vs 0.1696). It also has the higher PR-AUC (0.4016 vs 0.3879), the metric most relevant for the minority class. Both PR-AUC values are about 3.5 to 3.6 times the 0.111 that a random ranking would achieve on this test set.

Logistic Regression is more precise and ranks better. When Logistic Regression does say “yes”, it is right 68.79% of the time, compared with 60.23% for the tree. Its ROC-AUC of 0.7725 means that if one subscriber and one non-subscriber are picked at random, the model gives the subscriber the higher score about 77% of the time. Its low recall is not a lack of signal but a consequence of the **0.5 threshold**: its well-ranked probabilities rarely cross 0.5. Lowering the threshold (for example to 0.2) would trade some precision for much higher recall.

Which model should the bank prefer? For a calling campaign, a missed subscriber (false negative) is lost revenue, while an unnecessary call (false positive) costs only a few minutes of staff time. Recall is therefore more important than precision. At the default settings, the **Decision Tree is the more useful model**: each 100 clients it flags contain about 60 subscribers, compared with about 11 in a random list, and it finds twice as many subscribers as Logistic Regression. If the threshold can be tuned, Logistic Regression’s better ranking (higher ROC-AUC) could make it the stronger choice. Both models still miss most subscribers, which motivates the improvements in Chapter 10.

Chapter 9: Discussion and Business Insights

The analysis points to one clear targeting rule: call previous responders, older and younger clients, and loan-free clients first, and use a model tuned for recall rather than accuracy to rank the rest.

9.1 Consolidated Findings

The table below brings together the most important results from all fifteen objectives.

Table 9.1: Consolidated findings from all fifteen objectives

Area	Finding	Evidence
Data quality	No nulls or duplicates; "unknown" acts as missing data	0 nulls, 0 duplicates; 36,959 unknown *poutcome* (Obj 2)
Target	Strong class imbalance	11.70% yes vs 88.30% no (Obj 4)
Largest segment	Blue-collar workers	9,732 clients, 21.53% (Obj 5)
Age	U-shaped response	60+: 33.63%; below 30: 17.60%; 40–49: 9.12% (Obj 6)
Occupation	Students and retired most responsive	28.68% and 22.79%; blue-collar 7.27% (Obj 7)
Education	Rate rises with education	Tertiary 15.01% vs primary 8.63% (Obj 7)
Debt	Loan-free clients subscribe most	18.22% vs 6.07% with both loans (Obj 8)
Channel	Unknown contact type performs worst	4.07% vs 14.92% cellular (Obj 9)
Timing	Low-volume months convert best	March 51.99% (477 calls) vs May 6.72% (13,766 calls) (Obj 9)
History	Previous success is the strongest signal	64.73% vs 9.16% with no history (Obj 10)
Features	15 leak-free inputs, 59-dimensional vector	duration excluded (Obj 11–12)
Best recall	Decision Tree	20.84% recall, F1 0.3096, PR-AUC 0.4016 (Obj 14–15)
Best ranking	Logistic Regression	ROC-AUC 0.7725, precision 0.6879 (Obj 13, 15)
Baseline	Accuracy is misleading	Always "no" = 88.91% (Obj 15)

Several of these findings reinforce each other. The high rates among retired clients and the 60+ age group describe largely the same people. The rise in rate with education mirrors the difference between management and blue-collar occupations. Loan-free clients have the highest balances. And the Decision Tree, without being told any of this, chose the previous outcome, the contact type and the campaign month as its most important splits, confirming what the exploratory queries had shown.

At the same time, the analysis is **observational**. The bank chose whom to call and when, so rates for a month or a channel reflect both the client's inclination and the bank's selection. The very high March and September rates, for example, are based on a few hundred calls each and are best read as evidence

that focused campaigns aimed at warm leads work well, not as proof that March is inherently a better month.

9.2 Recommendations for the Marketing Team

Based on the findings, the following practical recommendations can be made:

1. **Prioritise past responders.** Clients whose previous campaign succeeded subscribed at 64.73%. They should be the first group contacted in any new campaign.
2. **Target by life stage.** Retired clients, clients aged 60 and above, and students respond far above average. Product messaging can be tailored to each group: secure savings for retirees, starter deposits for students.
3. **Consider financial capacity.** Clients without housing or personal loans and with higher balances are more than twice as likely to subscribe. Balance and loan status can be used to pre-screen call lists.
4. **Record the contact channel.** Calls with an unknown channel performed worst. Consistent recording of the channel would improve both campaign tracking and model quality.
5. **Favour focused campaigns over mass calling.** The busiest month had the lowest conversion. Smaller, well-targeted campaigns appear to produce much higher success rates per call.
6. **Limit repeated calls.** Some clients were called up to 63 times in one campaign. The Decision Tree used the campaign count as a split (around six contacts), suggesting that additional calls beyond a few attempts add little.
7. **Use the model as a ranking tool, with a tuned threshold.** Rather than using the default 0.5 cut-off, the bank should sort clients by predicted probability and call from the top of the list down to the number of calls its staff can make. The threshold should be chosen to balance the cost of a call against the value of a deposit.

9.3 Mapping to Course Outcomes

The project was designed to exercise the main topics of the INT315 syllabus. Table 9.2 shows where each course outcome is addressed.

Table 9.2: Mapping of project work to INT315 course outcomes

Course outcome	Syllabus unit	Where it is demonstrated in this project
CO1: Analyse the similarities and differences between Spark and Hadoop	Unit I	Chapter 2 (Sections 2.1–2.3, Table 2.1); in-memory caching and local-mode execution (Chapter 4)
CO2: Discover the features and functions of Scala for Apache Spark	Unit II	The same functional concepts are used through PySpark: immutable RDDs, lambda functions in `map` and `reduceByKey`, and higher-order transformations (Objective 5)
CO3: Discuss in-memory computation of RDDs for faster processing	Unit III	RDD creation, `map` and `reduceByKey`, narrow and wide transformations, lineage and caching (Chapter 2.4, Objective 5, Section 4.4)
CO4: Apply Spark MLlib for machine learning in a big data environment	Unit VI	Pipeline with StringIndexer, OneHotEncoder and VectorAssembler; Logistic Regression; Decision Tree; confusion matrix and evaluators (Objectives 11–15)
CO5: Analyse workflows and processing of big data using	Units IV, VI	End-to-end workflow (Figure 4.1); Spark SQL views, GROUP BY, ORDER BY, aggregate functions, LEFT

Course outcome	Syllabus unit	Where it is demonstrated in this project
Apache Spark		JOIN; RDD-to-DataFrame conversion (Objectives 1–10)
CO6: Apply Spark GraphX for graph processing	Unit III	Not applied, because the bank marketing data is tabular and has no natural graph structure; discussed as future scope in Chapter 10

Of the five practicals listed in the syllabus, this project directly demonstrates two: the use of MLlib (Objectives 11–15) and the use of Spark SQL (Objectives 7 and 10). The remaining three, GraphX, Spark Streaming and the Kafka producer–consumer model, are outside the scope of a static tabular dataset and are discussed as extensions in Chapter 10.

Chapter 10: Limitations and Future Scope

10.1 Limitations

- **Class imbalance was not corrected.** Both models were trained on the raw 88/12 class distribution with the default 0.5 threshold. This is the main reason that recall is low for both models.
- **No hyperparameter tuning.** The values `regParam=0.1` and `maxDepth=5` were chosen as reasonable defaults, not optimised. Other values could give better results.
- **Single random split.** Results come from one 80/20 split. A different seed would give slightly different numbers. Cross-validation would give a more reliable estimate with a confidence range.
- **Random rather than time-based evaluation.** The data is ordered by date from May 2008 to November 2010, but the split mixes all periods. In real use, the model would be trained on past campaigns and applied to future ones, which a date-based split would simulate more realistically.
- **Assumptions about available features.** The model assumes that the contact channel, day, month and attempt number are known before each call. If any of these are decided only during the campaign, the model's real-world performance could be lower.
- **Observational data.** Associations found in Objectives 6 to 10 cannot be interpreted as causes, because the bank selected whom to call and when.
- **Skewed numerical features were not transformed.** Balance, campaign and previous are heavily skewed. A log transformation or binning might help Logistic Regression.
- **Local mode only.** The program was run on a single machine. Its behaviour on a multi-node cluster, such as partitioning and shuffle costs, was not measured.
- **Dated dataset.** The data describes one Portuguese bank in 2008–2010, a period that included the global financial crisis. Client behaviour today may differ.

10.2 Future Scope

1. **Handle class imbalance.** Use the `weightCol` parameter of Spark's classifiers to give subscribers a higher weight (for example, inversely proportional to class frequency), or undersample the majority class in the training set.
2. **Tune the decision threshold.** Choose the probability threshold that maximises F1, or that meets a target recall, by evaluating several thresholds on a validation set. This alone would likely raise Logistic Regression's recall substantially.
3. **Hyperparameter tuning with cross-validation.** Use `ParamGridBuilder` with `CrossValidator` or `TrainValidationSplit` to search over `regParam`, `elasticNetParam`, `maxDepth` and `minInstancesPerNode`, optimising PR-AUC rather than accuracy.
4. **Ensemble methods.** Train `RandomForestClassifier` and `GBTClassifier`, which usually outperform a single tree, and compare them with the current models. Other algorithms in Unit VI, such as Linear SVM and Naive Bayes, could also be compared.
5. **Feature engineering.** Add a binary "previously contacted" flag derived from *pdays* = −1, log-transform *balance*, and bin *campaign* into ranges.
6. **Time-based validation.** Train on the earlier campaigns and test on the most recent months to estimate future performance.
7. **Use the extended dataset.** `bank-additional-full.csv` includes economic indicators such as the employment variation rate and the Euribor interest rate, which are known to be strong predictors.

8. **Real-time scoring with Spark Streaming and Kafka.** Following Unit V, a Kafka producer could publish new client records to a topic, and a Spark Structured Streaming job could load the saved pipeline and model to score clients as they arrive, writing the ranked call list to a database.
9. **Graph analytics with GraphX.** If data on relationships between clients (household links, referrals or shared branches) were available, GraphX could model clients as a graph and test whether subscribers cluster together.
10. **Cluster deployment.** Run the same code on a multi-node YARN or Kubernetes cluster with a larger, synthetic version of the data to measure scaling behaviour.

Chapter 11: Conclusion

This project used Apache Spark to analyse 45,211 bank marketing records and to predict term deposit subscription, completing all fifteen objectives in a single, reproducible PySpark program.

The first part of the work established that the data was structurally clean, with no nulls or duplicates, but that missing information was hidden behind the value “unknown” and that the target was strongly imbalanced, with only 11.70% subscribers. Using three Spark interfaces, the RDD API, the DataFrame API and Spark SQL, the analysis identified the client segments most likely to subscribe. The outcome of the previous campaign proved to be the strongest signal, with past responders subscribing at 64.73%. Clients aged 60 and above, students, retired clients, loan-free clients and clients with tertiary education also responded well above average, while blue-collar workers and clients contacted through an unknown channel responded least.

The second part built a Spark MLlib pipeline that encoded fifteen leak-free attributes into a 59-dimensional feature vector, deliberately excluding the call duration because it is only known after the call. The pipeline was fitted on the training data only, to avoid information leakage. Two classifiers from the syllabus were trained and evaluated on the same 9,048-record test set. The Decision Tree achieved 89.70% accuracy, 20.84% subscriber recall, an F1-score of 0.3096 and the higher PR-AUC of 0.4016. Logistic Regression achieved 89.50% accuracy and the higher ROC-AUC of 0.7725 and precision of 0.6879, but recalled only 9.67% of subscribers at the default threshold.

The most important lesson of the project is methodological. A model that always predicts “no” is already 88.91% accurate, so both classifiers look impressive by accuracy alone while missing most of the clients the bank actually wants to find. Evaluating with precision, recall, F1-score, PR-AUC and the confusion matrix gave a far more honest picture, and showed that the Decision Tree is more useful for a calling campaign at its default settings, while Logistic Regression would benefit most from threshold tuning.

From a course perspective, the project demonstrates the key ideas of INT315 in practice: lazy evaluation and in-memory caching, the MapReduce pattern through `map` and `reduceByKey`, conversion between RDDs and DataFrames, Spark SQL queries with grouping, ordering, aggregation and joins, and an end-to-end machine learning pipeline with Spark MLlib. Because Spark distributes this work automatically, the same code could be run without changes on a cluster processing millions of client records, which is exactly the setting in which a bank’s marketing analytics would operate. With the improvements outlined in Chapter 10, in particular class weighting, threshold tuning and ensemble models, this pipeline could become a practical tool for making marketing campaigns more efficient.

References

1. Moro, S., Cortez, P., and Rita, P. (2014). A data-driven approach to predict the success of bank telemarketing. *Decision Support Systems*, 62, 22–31.
2. Moro, S., Rita, P., and Cortez, P. (2012). Bank Marketing [Dataset]. UCI Machine Learning Repository. https://archive.ics.uci.edu/dataset/222/bank+marketing
3. Tandon, A., Ryza, S., Laserson, U., Owen, S., and Wills, J. (2022). *Advanced Analytics with PySpark: Patterns for Learning from Data at Scale Using Python and Spark*. O'Reilly Media (Shroff Publishers, Indian edition). Course text book.
4. Damji, J. S., Wenig, B., Das, T., and Lee, D. (2020). *Learning Spark: Lightning-Fast Data Analytics* (2nd ed.). O'Reilly Media. First edition by Karau, H., Konwinski, A., Wendell, P., and Zaharia, M. (2015). Course reference book.
5. Zaharia, M., Chowdhury, M., Das, T., Dave, A., Ma, J., McCauley, M., Franklin, M. J., Shenker, S., and Stoica, I. (2012). Resilient Distributed Datasets: A fault-tolerant abstraction for in-memory cluster computing. *Proceedings of the 9th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*.
6. Dean, J., and Ghemawat, S. (2004). MapReduce: Simplified data processing on large clusters. *Proceedings of the 6th Symposium on Operating Systems Design and Implementation (OSDI)*.
7. Armbrust, M., Xin, R. S., Lian, C., Huai, Y., Liu, D., Bradley, J. K., Meng, X., Kaftan, T., Franklin, M. J., Ghodsi, A., and Zaharia, M. (2015). Spark SQL: Relational data processing in Spark. *Proceedings of the 2015 ACM SIGMOD International Conference on Management of Data*.
8. Meng, X., Bradley, J., Yavuz, B., et al. (2016). MLlib: Machine learning in Apache Spark. *Journal of Machine Learning Research*, 17(34), 1–7.
9. Apache Software Foundation. Apache Spark documentation: RDD Programming Guide, Spark SQL Guide, and MLlib Guide (version 3.5). https://spark.apache.org/docs/latest/
10. Saito, T., and Rehmsmeier, M. (2015). The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets. *PLOS ONE*, 10(3), e0118432.
11. Lovely Professional University. INT315: Cluster Computing, course syllabus, session 2026–27.
12. Moro, S., Laureano, R., and Cortez, P. (2011). Using data mining for bank direct marketing: An application of the CRISP-DM methodology. In P. Novais et al. (Eds.), Proceedings of the European Simulation and Modelling Conference (ESM 2011), pp. 117–121, Guimarães, Portugal. EUROSIS. Original citation requested by the dataset authors in bank-names.txt.

Appendix A: Complete Console Output

The following is the complete output printed by the program in Appendix B when run on `bank-full.csv` in PySpark local mode. Spark's own log messages and progress bars are suppressed by `setLogLevel("ERROR")` and are therefore not shown. Because `randomSplit` samples within each data partition, the exact train/test counts in Objectives 11–15 can differ slightly on a machine where Spark creates a different number of input partitions; all other outputs are deterministic.

```
================================================================================
OBJECTIVE 1
================================================================================
PROBLEM STATEMENT: Load the bank marketing dataset and identify its number of
records, columns, and data types.

SOLUTION / OUTPUT:
Rows: 45211
Columns: 17
root
 |-- age: integer (nullable = true)
 |-- job: string (nullable = true)
 |-- marital: string (nullable = true)
 |-- education: string (nullable = true)
 |-- default: string (nullable = true)
 |-- balance: integer (nullable = true)
 |-- housing: string (nullable = true)
 |-- loan: string (nullable = true)
 |-- contact: string (nullable = true)
 |-- day: integer (nullable = true)
 |-- month: string (nullable = true)
 |-- duration: integer (nullable = true)
 |-- campaign: integer (nullable = true)
 |-- pdays: integer (nullable = true)
 |-- previous: integer (nullable = true)
 |-- poutcome: string (nullable = true)
 |-- y: string (nullable = true)

+---+-----------+-------+---------+-------+-------+-------+----+-------+---
+-----+--------+--------+-----+--------+--------+---+
|age|job        |marital|education|default|balance|housing|loan|contact|day|
month|duration|campaign|pdays|previous|poutcome|y  |
+---+-----------+-------+---------+-------+-------+-------+----+-------+---
+-----+--------+--------+-----+--------+--------+---+
|58 |management  |married|tertiary |no     |2143   |yes    |no  |unknown|5  |
may  |261     |1       |-1   |0       |unknown |no |
|44 |technician  |single |secondary|no     |29     |yes    |no  |unknown|5  |
may  |151     |1       |-1   |0       |unknown |no |
|33 |entrepreneur|married|secondary|no     |2      |yes    |yes |unknown|5  |
may  |76      |1       |-1   |0       |unknown |no |
|47 |blue-collar |married|unknown  |no     |1506   |yes    |no  |unknown|5  |
may  |92      |1       |-1   |0       |unknown |no |
|33 |unknown     |single |unknown  |no     |1      |no     |no  |unknown|5  |
may  |198     |1       |-1   |0       |unknown |no |
+---+-----------+-------+---------+-------+-------+-------+----+-------+---
+-----+--------+--------+-----+--------+--------+---+
only showing top 5 rows


SOLUTION IN WORDS: The dataset was loaded successfully with 45,211 records
and 17 columns.
--------------------------------------------------------------------------------

================================================================================
```

```
OBJECTIVE 2
================================================================================
PROBLEM STATEMENT: Identify null values, exact duplicate records, and unknown
categorical values.

SOLUTION / OUTPUT:
Null values:
+---+---+-------+---------+-------+-------+-------+----+-------+---+-----
+--------+--------+-----+--------+--------+---+
|age|job|marital|education|default|balance|housing|loan|contact|day|month|
duration|campaign|pdays|previous|poutcome|y  |
+---+---+-------+---------+-------+-------+-------+----+-------+---+-----
+--------+--------+-----+--------+--------+---+
|0  |0  |0      |0        |0      |0      |0      |0   |0      |0  |0    |0
|0       |0    |0       |0       |0  |
+---+---+-------+---------+-------+-------+-------+----+-------+---+-----
+--------+--------+-----+--------+--------+---+

Exact duplicate rows: 0
Unknown category counts:
+---+-------+---------+-------+-------+----+-------+-----+--------+---+
|job|marital|education|default|housing|loan|contact|month|poutcome|y  |
+---+-------+---------+-------+-------+----+-------+-----+--------+---+
|288|0      |1857     |0      |0      |0   |13020  |0    |36959   |0  |
+---+-------+---------+-------+-------+----+-------+-----+--------+---+


SOLUTION IN WORDS: The dataset contains 0 null cells and 0 exact duplicate
rows. 'unknown' is retained as a categorical value. The value pdays = -1
means not previously contacted.
--------------------------------------------------------------------------------

================================================================================
OBJECTIVE 3
================================================================================
PROBLEM STATEMENT: Calculate descriptive statistics for customer age,
balance, and numerical campaign attributes.

SOLUTION / OUTPUT:
+-------+-----------------+-----------------+-----------------
+-----------------+-----------------+-----------------+------------------+
|summary|age              |balance          |day              |duration
|campaign         |pdays            |previous         |
+-------+-----------------+-----------------+-----------------
+-----------------+-----------------+-----------------+------------------+
|count  |45211            |45211            |45211            |45211
|45211            |45211            |45211            |
|mean   |40.93621021432837 |1362.2720576850766|15.80641879188693|
258.1630797814691|2.763840658246887|40.19782796222158 |0.5803233726305546|
|stddev |10.618762040975405|3044.7658291685257|8.322476153044594|
257.5278122651706|3.098020883279192|100.12874599059828|2.303441044931218 |
|min    |18               |-8019            |1                |0
|1                |-1               |0                |
|25%    |33               |72               |8                |103
|1                |-1               |0                |
|50%    |39               |448              |16               |180
|2                |-1               |0                |
|75%    |48               |1426             |21               |319
|3                |-1               |0                |
|max    |95               |102127           |31               |4918
|63               |871              |275              |
+-------+-----------------+-----------------+-----------------
+-----------------+-----------------+-----------------+------------------+
```

```
SOLUTION IN WORDS: Customer ages range from 18 to 95 years, with an average
of 40.94 years. The average yearly account balance is 1362.27 euros.
-------------------------------------------------------------------------

=========================================================================
OBJECTIVE 4
=========================================================================
PROBLEM STATEMENT: Calculate subscriber and non-subscriber counts and
identify class imbalance.

SOLUTION / OUTPUT:
+---+-----+----------+
|  y|count|percentage|
+---+-----+----------+
| no|39922|      88.3|
|yes| 5289|      11.7|
+---+-----+----------+


SOLUTION IN WORDS: 5,289 customers (11.70%) subscribed, while 39,922
customers (88.30%) did not. The dataset is imbalanced because non-subscribers
form the majority. Labels are no = 0 and yes = 1.
-------------------------------------------------------------------------

=========================================================================
OBJECTIVE 5
=========================================================================
PROBLEM STATEMENT: Use RDD map and reduceByKey operations to count customers
belonging to each occupation.

SOLUTION / OUTPUT:
+-------------+--------------+
|job          |customer_count|
+-------------+--------------+
|blue-collar  |9732          |
|management   |9458          |
|technician   |7597          |
|admin.       |5171          |
|services     |4154          |
|retired      |2264          |
|self-employed|1579          |
|entrepreneur |1487          |
|unemployed   |1303          |
|housemaid    |1240          |
|student      |938           |
|unknown      |288           |
+-------------+--------------+


SOLUTION IN WORDS: RDD transformations counted the records for each job. The
largest occupation group is 'blue-collar', with 9,732 records.
-------------------------------------------------------------------------

=========================================================================
OBJECTIVE 6
=========================================================================
PROBLEM STATEMENT: Group customers by age and compare their term deposit
subscription rates.

SOLUTION / OUTPUT:
+-----------+---------+-----------+--------------------+
```

```
|age_group   |customers|subscribers|subscription_rate_pct|
+------------+---------+-----------+---------------------+
|60 and above|1784     |600        |33.6322869955157     |
|Below 30    |5273     |928        |17.59908970225678    |
|30-39       |18089    |1913       |10.57548786555365    |
|50-59       |8410     |785        |9.334126040428062    |
|40-49       |11655    |1063       |9.120549120549121    |
+------------+---------+-----------+---------------------+


SOLUTION IN WORDS: The '60 and above' age group has the highest observed
subscription rate of 33.63%.
--------------------------------------------------------------------------------

================================================================================
OBJECTIVE 7
================================================================================
PROBLEM STATEMENT: Use Spark SQL to compare subscription rates by occupation
and education level.

SOLUTION / OUTPUT:
Subscription rates by job:
+-------------+---------+---------------------+
|job          |customers|subscription_rate_pct|
+-------------+---------+---------------------+
|student      |938      |28.678038379530918   |
|retired      |2264     |22.791519434628977   |
|unemployed   |1303     |15.502686108979278   |
|management   |9458     |13.755550856417848   |
|admin.       |5171     |12.202668729452718   |
|self-employed|1579     |11.842938568714375   |
|unknown      |288      |11.805555555555555   |
|technician   |7597     |11.056996182703699   |
|services     |4154     |8.883004333172845    |
|housemaid    |1240     |8.790322580645162    |
|entrepreneur |1487     |8.271687962340282    |
|blue-collar  |9732     |7.274969173859433    |
+-------------+---------+---------------------+

Subscription rates by education:
+---------+---------+---------------------+
|education|customers|subscription_rate_pct|
+---------+---------+---------------------+
|tertiary |13301    |15.006390496955117   |
|unknown  |1857     |13.5702746365105     |
|secondary|23202    |10.559434531505904   |
|primary  |6851     |8.626477886439936    |
+---------+---------+---------------------+


SOLUTION IN WORDS: 'student' has the highest observed job-category
subscription rate at 28.68%. Among education categories, 'tertiary' has the
highest rate at 15.01%.
--------------------------------------------------------------------------------

================================================================================
OBJECTIVE 8
================================================================================
PROBLEM STATEMENT: Compare subscription rates for different combinations of
housing loans and personal loans.

SOLUTION / OUTPUT:
+-------+----+---------+--------------+---------------------+
```

```
|housing|loan|customers|average_balance|subscription_rate_pct|
+-------+----+---------+---------------+---------------------+
|no     |no  |17204    |1737.61        |18.22250639386189    |
|yes    |no  |20763    |1256.41        |8.043153686846795    |
|no     |yes |2877     |752.71         |7.6120959332638165   |
|yes    |yes |4367     |788.54         |6.068239065720174    |
+-------+----+---------+---------------+---------------------+


SOLUTION IN WORDS: Customers with housing loan = 'no' and personal loan =
'no' have the highest observed subscription rate of 18.22%. This is an
association, not proof of causation.
--------------------------------------------------------------------------------

================================================================================
OBJECTIVE 9
================================================================================
PROBLEM STATEMENT: Compare historical subscription rates across contact
methods and campaign months.

SOLUTION / OUTPUT:
Contact methods:
+---------+---------+---------------------+
|contact  |customers|subscription_rate_pct|
+---------+---------+---------------------+
|cellular |29285    |14.918900460986853   |
|telephone|2906     |13.420509291121817   |
|unknown  |13020    |4.070660522273426    |
+---------+---------+---------------------+

Campaign months:
+-----+---------+---------------------+
|month|customers|subscription_rate_pct|
+-----+---------+---------------------+
|mar  |477      |51.99161425576519    |
|dec  |214      |46.728971962616825   |
|sep  |579      |46.45941278065631    |
|oct  |738      |43.766937669376695   |
|apr  |2932     |19.679399727148706   |
|feb  |2649     |16.64779161947905    |
|aug  |6247     |11.013286377461181   |
|jun  |5341     |10.222804718217562   |
|nov  |3970     |10.151133501259446   |
|jan  |1403     |10.121168923734853   |
|jul  |6895     |9.09354604786077     |
|may  |13766    |6.719453726572715    |
+-----+---------+---------------------+


SOLUTION IN WORDS: 'cellular' has the highest observed contact-method
subscription rate at 14.92%. The highest monthly rate is 51.99% in 'mar',
based on 477 records. Different group sizes and customer selection must be
considered when interpreting these rates.
--------------------------------------------------------------------------------

================================================================================
OBJECTIVE 10
================================================================================
PROBLEM STATEMENT: Use an SQL LEFT JOIN to attach descriptions of previous
campaign outcomes and compare current subscription rates.

SOLUTION / OUTPUT:
+--------+--------------------------+---------+---------------------+
```

```
|poutcome|outcome_description        |customers|subscription_rate_pct|
+--------+---------------------------+---------+---------------------+
|success |Previous campaign succeeded|1511     |64.72534745201854    |
|other   |Other previous outcome     |1840     |16.684782608695652   |
|failure |Previous campaign failed   |4901     |12.609671495613139   |
|unknown |Previous outcome unknown   |36959    |9.161503287426607    |
+--------+---------------------------+---------+---------------------+


SOLUTION IN WORDS: The LEFT JOIN added readable outcome descriptions. The
previous outcome 'success' is associated with the highest observed current
subscription rate of 64.73%.
--------------------------------------------------------------------------------

================================================================================
OBJECTIVE 11
================================================================================
PROBLEM STATEMENT: Select prediction features and split the data into
approximately 80% training and 20% testing.

SOLUTION / OUTPUT:
Categorical inputs: ['job', 'marital', 'education', 'default', 'housing',
'loan', 'contact', 'month', 'poutcome']
Numerical inputs: ['age', 'balance', 'day', 'campaign', 'pdays', 'previous']
Training records: 36163
Testing records: 9048
Training label distribution:
+-----+-----+
|label|count|
+-----+-----+
|  0.0|31877|
|  1.0| 4286|
+-----+-----+

Testing label distribution:
+-----+-----+
|label|count|
+-----+-----+
|  0.0| 8045|
|  1.0| 1003|
+-----+-----+


SOLUTION IN WORDS: The dataset was split into 36,163 training records and
9,048 testing records. There are 15 input columns: 9 categorical and 6
numerical. The target y and post-call duration are excluded. This random
split is a coursework holdout experiment, not a time-based evaluation of
future campaigns.
--------------------------------------------------------------------------------

================================================================================
OBJECTIVE 12
================================================================================
PROBLEM STATEMENT: Encode categorical variables and combine all inputs into a
feature vector using a Spark ML pipeline.

SOLUTION / OUTPUT:
+--------------------------------------------------------------------------------
-----------------------------+-----+
|features
|label|
+--------------------------------------------------------------------------------
-----------------------------+-----+
```

```
|(59,[0,1,2,3,4,9,21,25,28,32,34,37,42,54],[36.0,227.0,9.0,1.0,-
1.0,1.0,1.0,1.0,1.0,1.0,1.0,1.0,1.0,1.0]) |0.0  |
|(59,[0,1,2,3,4,9,21,25,28,32,34,37,42,54],[46.0,509.0,17.0,1.0,-
1.0,1.0,1.0,1.0,1.0,1.0,1.0,1.0,1.0,1.0])|0.0  |
|(59,[0,1,2,3,4,9,21,25,28,32,34,38,44,54],[57.0,987.0,16.0,2.0,-
1.0,1.0,1.0,1.0,1.0,1.0,1.0,1.0,1.0,1.0])|0.0  |
|(59,[0,1,2,3,4,9,21,25,28,32,34,38,41,54],[57.0,1.0,15.0,10.0,-
1.0,1.0,1.0,1.0,1.0,1.0,1.0,1.0,1.0,1.0]) |0.0  |
|(59,[0,1,2,3,4,9,21,25,28,32,35,37,42,54],[46.0,505.0,18.0,4.0,-
1.0,1.0,1.0,1.0,1.0,1.0,1.0,1.0,1.0,1.0])|0.0  |
+-----------------------------------------------------------------------------
----------------------------+-----+
only showing top 5 rows


SOLUTION IN WORDS: StringIndexer and OneHotEncoder encoded the categorical
variables. VectorAssembler combined the inputs into a feature vector with 59
positions. Preprocessing was fitted only on the training set to prevent
information leakage from the test set.
-------------------------------------------------------------------------------

================================================================================
OBJECTIVE 13
================================================================================
PROBLEM STATEMENT: Train a Logistic Regression classifier to predict term
deposit subscriptions.

SOLUTION / OUTPUT:
+-----+----------+----------------------------------------+
|label|prediction|probability                             |
+-----+----------+----------------------------------------+
|0.0  |0.0       |[0.9052937360151655,0.0947062639848345] |
|0.0  |0.0       |[0.8982257277456548,0.10177427225434521]|
|1.0  |0.0       |[0.5419254685381983,0.45807453146180166]|
|0.0  |0.0       |[0.8895839964108272,0.1104160035891728] |
|0.0  |0.0       |[0.9381809866173324,0.0618190133826676] |
|0.0  |0.0       |[0.9409496168492809,0.05905038315071909]|
|0.0  |0.0       |[0.9578315634780667,0.04216843652193325]|
|0.0  |0.0       |[0.8388972131258935,0.1611027868741065] |
|0.0  |0.0       |[0.8484088820523576,0.15159111794764235]|
|0.0  |0.0       |[0.8757437125099634,0.12425628749003659]|
+-----+----------+----------------------------------------+
only showing top 10 rows


SOLUTION IN WORDS: Logistic Regression achieved 89.50% test accuracy and
9.67% subscriber recall. It correctly identified 97 actual subscribers. The
probability vector shows [P(no), P(yes)].
-------------------------------------------------------------------------------

================================================================================
OBJECTIVE 14
================================================================================
PROBLEM STATEMENT: Train a Decision Tree classifier using the same training
and testing data.

SOLUTION / OUTPUT:
+-----+----------+----------------------------------------+
|label|prediction|probability                             |
+-----+----------+----------------------------------------+
|0.0  |0.0       |[0.960578313253012,0.039421686746987955]|
|0.0  |0.0       |[0.8859463575598613,0.11405364244013876]|
|1.0  |1.0       |[0.4369747899159664,0.5630252100840336] |
```

```
|0.0  |0.0       |[0.8859463575598613,0.11405364244013876]|
|0.0  |0.0       |[0.960578313253012,0.039421686746987955]|
|0.0  |0.0       |[0.960578313253012,0.039421686746987955]|
|0.0  |0.0       |[0.960578313253012,0.039421686746987955]|
|0.0  |0.0       |[0.8859463575598613,0.11405364244013876]|
|0.0  |0.0       |[0.8859463575598613,0.11405364244013876]|
|0.0  |0.0       |[0.8859463575598613,0.11405364244013876]|
+-----+----------+----------------------------------------+
only showing top 10 rows


SOLUTION IN WORDS: The Decision Tree achieved 89.70% test accuracy and 20.84%
subscriber recall. It correctly identified 209 actual subscribers.
--------------------------------------------------------------------------------

================================================================================
OBJECTIVE 15
================================================================================
PROBLEM STATEMENT: Compare both models using accuracy, ROC-AUC, PR-AUC,
subscriber precision, recall, F1, and confusion matrices.

SOLUTION / OUTPUT:
+-------------------+--------+-------+------+-------------+----------+------+
|model              |accuracy|roc_auc|pr_auc|yes_precision|yes_recall|yes_f1|
+-------------------+--------+-------+------+-------------+----------+------+
|Logistic Regression|0.895   |0.7725 |0.3879|0.6879       |0.0967    |0.1696|
|Decision Tree      |0.897   |0.6887 |0.4016|0.6023       |0.2084    |0.3096|
+-------------------+--------+-------+------+-------------+----------+------+


Confusion matrix: Logistic Regression
Rows = actual label; columns = predicted label
0 = No subscription; 1 = Subscription
+-----+----+---+
|label| 0.0|1.0|
+-----+----+---+
|  0.0|8001| 44|
|  1.0| 906| 97|
+-----+----+---+

TN: 8001 FP: 44 FN: 906 TP: 97

Confusion matrix: Decision Tree
Rows = actual label; columns = predicted label
0 = No subscription; 1 = Subscription
+-----+----+---+
|label| 0.0|1.0|
+-----+----+---+
|  0.0|7907|138|
|  1.0| 794|209|
+-----+----+---+

TN: 7907 FP: 138 FN: 794 TP: 209

Always predicting NO gives accuracy: 88.91%

SOLUTION IN WORDS: Decision Tree has the highest subscriber recall among the
compared models at 20.84%. Logistic Regression has the highest ROC-AUC at
0.7725. Always predicting 'no' already gives 88.91% accuracy, so accuracy
alone is insufficient. Precision, recall, F1 and PR-AUC help assess
performance on subscribers.
--------------------------------------------------------------------------------
```

```
================================================================================
ALL 15 OBJECTIVES COMPLETED SUCCESSFULLY.
Each objective includes its problem, output and conclusion.
================================================================================
```

Appendix B: Complete Source Code

The complete PySpark program is reproduced below exactly as submitted. It is wrapped in `exec(r'''...''')` so that it can be pasted into the PySpark shell as a single statement.

```
exec(r'''
# ============================================================
# INT315: BANK MARKETING ANALYSIS AND DEPOSIT PREDICTION
# Complete standalone code for all 15 objectives
# ============================================================

from pyspark.sql import SparkSession
from pyspark.sql import functions as F
from pyspark.ml import Pipeline
from pyspark.ml.feature import (
    StringIndexer,
    OneHotEncoder,
    VectorAssembler
)
from pyspark.ml.classification import (
    LogisticRegression,
    DecisionTreeClassifier
)
from pyspark.ml.evaluation import (
    BinaryClassificationEvaluator,
    MulticlassClassificationEvaluator
)

spark = (
    SparkSession.builder
    .appName("BankMarketingProject")
    .getOrCreate()
)

spark.sparkContext.setLogLevel("ERROR")
spark.conf.set("spark.sql.shuffle.partitions", "8")


def heading(number, problem):
    print("\n" + "=" * 75)
    print("OBJECTIVE", number)
    print("=" * 75)
    print("PROBLEM STATEMENT:", problem)
    print("\nSOLUTION / OUTPUT:")


def conclusion(text):
    print("\nSOLUTION IN WORDS:", text)
    print("-" * 75)


# ============================================================
# OBJECTIVE 1
# Problem statement: Load the dataset and inspect its structure.
# ============================================================

heading(
    1,
    "Load the bank marketing dataset and identify its "
    "number of records, columns, and data types."
)

path = (
```

```
    "file:///mnt/c/Users/krish/OneDrive/Desktop/"
    "acedemic notes/cluster_computing ca2/bank/bank-full.csv"
)

df = (
    spark.read
    .option("header", True)
    .option("sep", ";")
    .option("inferSchema", True)
    .option("mode", "FAILFAST")
    .csv(path)
    .cache()
)

total_rows = df.count()

print("Rows:", total_rows)
print("Columns:", len(df.columns))
df.printSchema()
df.show(5, truncate=False)

conclusion(
    f"The dataset was loaded successfully with "
    f"{total_rows:,} records and {len(df.columns)} columns."
)


# ============================================================
# OBJECTIVE 2
# Problem statement: Check nulls, duplicates and unknown values.
# ============================================================

heading(
    2,
    "Identify null values, exact duplicate records, "
    "and unknown categorical values."
)

null_counts = df.select([
    F.sum(F.col(c).isNull().cast("int")).alias(c)
    for c in df.columns
])

print("Null values:")
null_counts.show(truncate=False)

total_nulls = sum(
    value or 0
    for value in null_counts.first().asDict().values()
)

duplicate_count = total_rows - df.dropDuplicates().count()
print("Exact duplicate rows:", duplicate_count)

string_columns = [
    name for name, dtype in df.dtypes
    if dtype == "string"
]

print("Unknown category counts:")

df.select([
    F.sum(
```

```
        F.when(F.col(c) == "unknown", 1).otherwise(0)
    ).alias(c)
    for c in string_columns
]).show(truncate=False)

conclusion(
    f"The dataset contains {total_nulls:,} null cells and "
    f"{duplicate_count:,} exact duplicate rows. "
    "'unknown' is retained as a categorical value. "
    "The value pdays = -1 means not previously contacted."
)


# ============================================================
# OBJECTIVE 3
# Problem statement: Calculate numerical descriptive statistics.
# ============================================================

heading(
    3,
    "Calculate descriptive statistics for customer age, "
    "balance, and numerical campaign attributes."
)

numeric_columns = [
    "age", "balance", "day", "duration",
    "campaign", "pdays", "previous"
]

df.select(numeric_columns).summary(
    "count", "mean", "stddev",
    "min", "25%", "50%", "75%", "max"
).show(truncate=False)

stats = df.agg(
    F.min("age").alias("min_age"),
    F.max("age").alias("max_age"),
    F.avg("age").alias("avg_age"),
    F.avg("balance").alias("avg_balance")
).first()

conclusion(
    f"Customer ages range from {stats['min_age']} to "
    f"{stats['max_age']} years, with an average of "
    f"{stats['avg_age']:.2f} years. The average yearly "
    f"account balance is {stats['avg_balance']:.2f} euros."
)


# ============================================================
# OBJECTIVE 4
# Problem statement: Analyse target distribution and imbalance.
# ============================================================

heading(
    4,
    "Calculate subscriber and non-subscriber counts "
    "and identify class imbalance."
)

df.groupBy("y").count().withColumn(
    "percentage",
    F.round(F.col("count") * 100.0 / total_rows, 2)
```

```
).orderBy("y").show()

yes_count = df.filter(F.col("y") == "yes").count()
no_count = df.filter(F.col("y") == "no").count()

analysis_df = (
    df.filter(F.col("y").isin("yes", "no"))
    .withColumn(
        "label",
        F.when(F.col("y") == "yes", 1.0).otherwise(0.0)
    )
)

conclusion(
    f"{yes_count:,} customers "
    f"({yes_count / total_rows * 100:.2f}%) subscribed, "
    f"while {no_count:,} customers "
    f"({no_count / total_rows * 100:.2f}%) did not. "
    "The dataset is imbalanced because non-subscribers "
    "form the majority. Labels are no = 0 and yes = 1."
)


# ============================================================
# OBJECTIVE 5
# Problem statement: Count occupations using RDD operations.
# ============================================================

heading(
    5,
    "Use RDD map and reduceByKey operations to count "
    "customers belonging to each occupation."
)

job_counts_rdd = (
    df.select("job").rdd
    .map(lambda row: (row["job"], 1))
    .reduceByKey(lambda a, b: a + b)
)

job_counts_df = job_counts_rdd.toDF(
    ["job", "customer_count"]
).orderBy(
    F.desc("customer_count"), "job"
)

job_counts_df.show(truncate=False)
largest_job = job_counts_df.first()

conclusion(
    "RDD transformations counted the records for each job. "
    f"The largest occupation group is "
    f"'{largest_job['job']}', with "
    f"{largest_job['customer_count']:,} records."
)


# ============================================================
# OBJECTIVE 6
# Problem statement: Compare subscription rates by age group.
# ============================================================

heading(
```

```
    6,
    "Group customers by age and compare their "
    "term deposit subscription rates."
)

age_df = analysis_df.withColumn(
    "age_group",
    F.when(F.col("age") < 30, "Below 30")
    .when(F.col("age") < 40, "30-39")
    .when(F.col("age") < 50, "40-49")
    .when(F.col("age") < 60, "50-59")
    .otherwise("60 and above")
)

age_summary = age_df.groupBy("age_group").agg(
    F.count("*").alias("customers"),
    F.sum("label").cast("long").alias("subscribers"),
    (F.avg("label") * 100).alias("subscription_rate_pct")
).orderBy(
    F.desc("subscription_rate_pct")
)

age_summary.show(truncate=False)
best_age = age_summary.first()

conclusion(
    f"The '{best_age['age_group']}' age group has the "
    f"highest observed subscription rate of "
    f"{best_age['subscription_rate_pct']:.2f}%."
)


# ============================================================
# OBJECTIVE 7
# Problem statement: Analyse jobs and education using Spark SQL.
# ============================================================

heading(
    7,
    "Use Spark SQL to compare subscription rates "
    "by occupation and education level."
)

analysis_df.createOrReplaceTempView("bank")

job_rates = spark.sql("""
    SELECT
        job,
        COUNT(*) AS customers,
        AVG(label) * 100 AS subscription_rate_pct
    FROM bank
    GROUP BY job
    ORDER BY subscription_rate_pct DESC
""")

education_rates = spark.sql("""
    SELECT
        education,
        COUNT(*) AS customers,
        AVG(label) * 100 AS subscription_rate_pct
    FROM bank
    GROUP BY education
    ORDER BY subscription_rate_pct DESC
```

```
""")

print("Subscription rates by job:")
job_rates.show(truncate=False)

print("Subscription rates by education:")
education_rates.show(truncate=False)

best_job = job_rates.first()
best_education = education_rates.first()

conclusion(
    f"'{best_job['job']}' has the highest observed "
    f"job-category subscription rate at "
    f"{best_job['subscription_rate_pct']:.2f}%. "
    f"Among education categories, "
    f"'{best_education['education']}' has the highest "
    f"rate at {best_education['subscription_rate_pct']:.2f}%."
)


# ============================================================
# OBJECTIVE 8
# Problem statement: Analyse loan status and subscriptions.
# ============================================================

heading(
    8,
    "Compare subscription rates for different combinations "
    "of housing loans and personal loans."
)

loan_summary = analysis_df.groupBy("housing", "loan").agg(
    F.count("*").alias("customers"),
    F.round(F.avg("balance"), 2).alias("average_balance"),
    (F.avg("label") * 100).alias("subscription_rate_pct")
).orderBy(
    F.desc("subscription_rate_pct")
)

loan_summary.show(truncate=False)
best_loan = loan_summary.first()

conclusion(
    f"Customers with housing loan = '{best_loan['housing']}' "
    f"and personal loan = '{best_loan['loan']}' have the "
    f"highest observed subscription rate of "
    f"{best_loan['subscription_rate_pct']:.2f}%. "
    "This is an association, not proof of causation."
)


# ============================================================
# OBJECTIVE 9
# Problem statement: Analyse contact methods and campaign months.
# ============================================================

heading(
    9,
    "Compare historical subscription rates across "
    "contact methods and campaign months."
)
```

```
contact_summary = analysis_df.groupBy("contact").agg(
    F.count("*").alias("customers"),
    (F.avg("label") * 100).alias("subscription_rate_pct")
).orderBy(
    F.desc("subscription_rate_pct")
)

month_summary = analysis_df.groupBy("month").agg(
    F.count("*").alias("customers"),
    (F.avg("label") * 100).alias("subscription_rate_pct")
).orderBy(
    F.desc("subscription_rate_pct")
)

print("Contact methods:")
contact_summary.show(truncate=False)

print("Campaign months:")
month_summary.show(truncate=False)

best_contact = contact_summary.first()
best_month = month_summary.first()

conclusion(
    f"'{best_contact['contact']}' has the highest observed "
    f"contact-method subscription rate at "
    f"{best_contact['subscription_rate_pct']:.2f}%. "
    f"The highest monthly rate is "
    f"{best_month['subscription_rate_pct']:.2f}% in "
    f"'{best_month['month']}', based on "
    f"{best_month['customers']:,} records. "
    "Different group sizes and customer selection "
    "must be considered when interpreting these rates."
)


# ============================================================
# OBJECTIVE 10
# Problem statement: Analyse previous outcomes using an SQL join.
# ============================================================

heading(
    10,
    "Use an SQL LEFT JOIN to attach descriptions of previous "
    "campaign outcomes and compare current subscription rates."
)

outcome_lookup = spark.createDataFrame(
    [
        ("success", "Previous campaign succeeded"),
        ("failure", "Previous campaign failed"),
        ("other", "Other previous outcome"),
        ("unknown", "Previous outcome unknown")
    ],
    ["outcome_code", "outcome_description"]
)

outcome_lookup.createOrReplaceTempView("outcome_lookup")

outcome_summary = spark.sql("""
    SELECT
        b.poutcome,
        o.outcome_description,
```

```
        COUNT(*) AS customers,
        AVG(b.label) * 100 AS subscription_rate_pct
    FROM bank b
    LEFT JOIN outcome_lookup o
        ON b.poutcome = o.outcome_code
    GROUP BY b.poutcome, o.outcome_description
    ORDER BY subscription_rate_pct DESC
""")

outcome_summary.show(truncate=False)
best_outcome = outcome_summary.first()

conclusion(
    "The LEFT JOIN added readable outcome descriptions. "
    f"The previous outcome '{best_outcome['poutcome']}' "
    f"is associated with the highest observed current "
    f"subscription rate of "
    f"{best_outcome['subscription_rate_pct']:.2f}%."
)


# ============================================================
# OBJECTIVE 11
# Problem statement: Select features and split the dataset.
# ============================================================

heading(
    11,
    "Select prediction features and split the data "
    "into approximately 80% training and 20% testing."
)

categorical_features = [
    "job", "marital", "education", "default",
    "housing", "loan", "contact", "month", "poutcome"
]

numeric_features = [
    "age", "balance", "day",
    "campaign", "pdays", "previous"
]

# Duration is excluded because it is known after the call.
# The target y must not be included as an input feature.
# Contact date, method and attempt count are assumed known
# at prediction time.

model_df = (
    analysis_df
    .select(
        *(categorical_features + numeric_features + ["label"])
    )
    .na.fill("unknown", subset=categorical_features)
    .na.drop(subset=numeric_features)
)

train_df, test_df = model_df.randomSplit(
    [0.8, 0.2],
    seed=42
)

train_df = train_df.cache()
test_df = test_df.cache()
```

```
train_count = train_df.count()
test_count = test_df.count()

print("Categorical inputs:", categorical_features)
print("Numerical inputs:", numeric_features)
print("Training records:", train_count)
print("Testing records:", test_count)

print("Training label distribution:")
train_df.groupBy("label").count().orderBy("label").show()

print("Testing label distribution:")
test_df.groupBy("label").count().orderBy("label").show()

conclusion(
    f"The dataset was split into {train_count:,} training "
    f"records and {test_count:,} testing records. "
    "There are 15 input columns: 9 categorical and 6 numerical. "
    "The target y and post-call duration are excluded. "
    "This random split is a coursework holdout experiment, "
    "not a time-based evaluation of future campaigns."
)


# ============================================================
# OBJECTIVE 12
# Problem statement: Encode and assemble model features.
# ============================================================

heading(
    12,
    "Encode categorical variables and combine all inputs "
    "into a feature vector using a Spark ML pipeline."
)

indexers = [
    StringIndexer(
        inputCol=c,
        outputCol=c + "_index",
        handleInvalid="keep"
    )
    for c in categorical_features
]

encoder = OneHotEncoder(
    inputCols=[
        c + "_index" for c in categorical_features
    ],
    outputCols=[
        c + "_encoded" for c in categorical_features
    ],
    handleInvalid="keep"
)

assembler = VectorAssembler(
    inputCols=(
        numeric_features
        + [c + "_encoded" for c in categorical_features]
    ),
    outputCol="features"
)
```

```
preprocessing_pipeline = Pipeline(
    stages=indexers + [encoder, assembler]
)

# Fit preprocessing only on training data.
preprocessing_model = preprocessing_pipeline.fit(train_df)

train_ready = (
    preprocessing_model.transform(train_df)
    .select("features", "label")
    .cache()
)

test_ready = (
    preprocessing_model.transform(test_df)
    .select("features", "label")
    .cache()
)

train_ready.show(5, truncate=False)

vector_size = (
    train_ready.select("features").first()["features"].size
)

conclusion(
    "StringIndexer and OneHotEncoder encoded the categorical "
    "variables. VectorAssembler combined the inputs into "
    f"a feature vector with {vector_size} positions. "
    "Preprocessing was fitted only on the training set "
    "to prevent information leakage from the test set."
)


# ============================================================
# COMMON MODEL EVALUATION HELPERS
# ============================================================

accuracy_evaluator = MulticlassClassificationEvaluator(
    labelCol="label",
    predictionCol="prediction",
    metricName="accuracy"
)

roc_evaluator = BinaryClassificationEvaluator(
    labelCol="label",
    rawPredictionCol="probability",
    metricName="areaUnderROC"
)

pr_evaluator = BinaryClassificationEvaluator(
    labelCol="label",
    rawPredictionCol="probability",
    metricName="areaUnderPR"
)


def get_metrics(name, predictions):
    counts = {
        (int(row["label"]), int(row["prediction"])): row["count"]
        for row in predictions.groupBy(
            "label", "prediction"
        ).count().collect()
```

```
    }

    tn = counts.get((0, 0), 0)
    fp = counts.get((0, 1), 0)
    fn = counts.get((1, 0), 0)
    tp = counts.get((1, 1), 0)

    precision = tp / (tp + fp) if tp + fp else 0.0
    recall = tp / (tp + fn) if tp + fn else 0.0

    f1 = (
        2 * precision * recall / (precision + recall)
        if precision + recall else 0.0
    )

    return {
        "model": name,
        "accuracy": accuracy_evaluator.evaluate(predictions),
        "roc_auc": roc_evaluator.evaluate(predictions),
        "pr_auc": pr_evaluator.evaluate(predictions),
        "yes_precision": precision,
        "yes_recall": recall,
        "yes_f1": f1,
        "TN": tn,
        "FP": fp,
        "FN": fn,
        "TP": tp
    }


# ============================================================
# OBJECTIVE 13
# Problem statement: Train and evaluate Logistic Regression.
# ============================================================

heading(
    13,
    "Train a Logistic Regression classifier to predict "
    "term deposit subscriptions."
)

lr = LogisticRegression(
    featuresCol="features",
    labelCol="label",
    maxIter=100,
    regParam=0.1,
    standardization=True
)

lr_model = lr.fit(train_ready)
lr_predictions = lr_model.transform(test_ready).cache()

lr_predictions.select(
    "label", "prediction", "probability"
).show(10, truncate=False)

lr_metrics = get_metrics(
    "Logistic Regression",
    lr_predictions
)

conclusion(
    f"Logistic Regression achieved "
```

```
    f"{lr_metrics['accuracy'] * 100:.2f}% test accuracy "
    f"and {lr_metrics['yes_recall'] * 100:.2f}% subscriber "
    f"recall. It correctly identified "
    f"{lr_metrics['TP']:,} actual subscribers. "
    "The probability vector shows [P(no), P(yes)]."
)


# ============================================================
# OBJECTIVE 14
# Problem statement: Train and evaluate a Decision Tree.
# ============================================================

heading(
    14,
    "Train a Decision Tree classifier using the same "
    "training and testing data."
)

dt = DecisionTreeClassifier(
    featuresCol="features",
    labelCol="label",
    maxDepth=5,
    seed=42
)

dt_model = dt.fit(train_ready)
dt_predictions = dt_model.transform(test_ready).cache()

dt_predictions.select(
    "label", "prediction", "probability"
).show(10, truncate=False)

dt_metrics = get_metrics(
    "Decision Tree",
    dt_predictions
)

conclusion(
    f"The Decision Tree achieved "
    f"{dt_metrics['accuracy'] * 100:.2f}% test accuracy "
    f"and {dt_metrics['yes_recall'] * 100:.2f}% subscriber "
    f"recall. It correctly identified "
    f"{dt_metrics['TP']:,} actual subscribers."
)


# ============================================================
# OBJECTIVE 15
# Problem statement: Compare classifiers and baseline accuracy.
# ============================================================

heading(
    15,
    "Compare both models using accuracy, ROC-AUC, PR-AUC, "
    "subscriber precision, recall, F1, and confusion matrices."
)

metric_columns = [
    "model", "accuracy", "roc_auc", "pr_auc",
    "yes_precision", "yes_recall", "yes_f1"
]
```

```
metrics_list = [lr_metrics, dt_metrics]

comparison_df = spark.createDataFrame(
    [
        tuple(m[c] for c in metric_columns)
        for m in metrics_list
    ],
    metric_columns
)

comparison_df.select(
    "model",
    *[
        F.round(F.col(c), 4).alias(c)
        for c in metric_columns
        if c != "model"
    ]
).show(truncate=False)

for metrics, predictions in [
    (lr_metrics, lr_predictions),
    (dt_metrics, dt_predictions)
]:
    print("\nConfusion matrix:", metrics["model"])
    print("Rows = actual label; columns = predicted label")
    print("0 = No subscription; 1 = Subscription")

    predictions.groupBy("label").pivot(
        "prediction", [0.0, 1.0]
    ).count().fillna(0).orderBy("label").show()

    print(
        "TN:", metrics["TN"],
        "FP:", metrics["FP"],
        "FN:", metrics["FN"],
        "TP:", metrics["TP"]
    )

baseline_accuracy = (
    test_df.filter(F.col("label") == 0.0).count()
    / test_count
)

best_recall = max(
    metrics_list,
    key=lambda item: item["yes_recall"]
)

best_auc = max(
    metrics_list,
    key=lambda item: item["roc_auc"]
)

print(
    "\nAlways predicting NO gives accuracy:",
    f"{baseline_accuracy * 100:.2f}%"
)

conclusion(
    f"{best_recall['model']} has the highest subscriber "
    f"recall among the compared models at "
    f"{best_recall['yes_recall'] * 100:.2f}%. "
    f"{best_auc['model']} has the highest ROC-AUC "
```

```
    f"at {best_auc['roc_auc']:.4f}. "
    f"Always predicting 'no' already gives "
    f"{baseline_accuracy * 100:.2f}% accuracy, "
    "so accuracy alone is insufficient. Precision, recall, "
    "F1 and PR-AUC help assess performance on subscribers."
)


# Release cached memory without closing the PySpark shell.
lr_predictions.unpersist()
dt_predictions.unpersist()
train_ready.unpersist()
test_ready.unpersist()
train_df.unpersist()
test_df.unpersist()
df.unpersist()

print("\n" + "=" * 75)
print("ALL 15 OBJECTIVES COMPLETED SUCCESSFULLY.")
print("Each objective includes its problem, output and conclusion.")
print("=" * 75)
''')
```